

SLOT8:

Handlebars

Tóm tắt nội dung

HandleBars:

22.1 What Is Handlebars?

22.2 Understand What Handlebars Does And How To Use It With Node And Express

23.3 Templating With Handlebars

1. What is Handlebars?

- **Handlebars.js** là một **template engine** dùng để **tạo HTML động** bằng cách kết hợp **template** với **dữ liệu**.
- Nó sử dụng cú pháp **biến thay thế ({{{}}})** để chèn dữ liệu vào template.
- Ví dụ:

```
handlebars
```

```
<h1>Hello, {{{name}}}!</h1>
```

Với dữ liệu { name: 'Alice' }, kết quả sẽ là:

```
html
```

```
<h1>Hello, Alice!</h1>
```

2. Understand What Handlebars Does and How to Use It with Node and Express

- **Mục đích chính:** Giúp bạn **tách biệt logic xử lý** (Node.js/Express) và **giao diện người dùng (HTML)**.
- **Tích hợp với Express:**
 1. Cài đặt:

```
bash
```

```
npm install express express-handlebars
```

2. Cấu hình Express để dùng Handlebars:

```
javascript
```

```
const express = require('express');  
const exphbs = require('express-handlebars');
```

```
const app = express();
```

```
app.engine('handlebars', exphbs());  
app.set('view engine', 'handlebars');
```

3. Tạo file view (ví dụ home.handlebars) trong thư mục views:

```
handlebars
```

```
<h1>Welcome, {{username}}!</h1>
```

4. Render trong route:

```
javascript

app.get('/', (req, res) => {
  res.render('home', { username: 'Alice' });
});
```

3. Templating with Handlebars

- **Dữ liệu động:** Chèn dữ liệu vào HTML để dàng bằng {{variable}}.
- **Cấu trúc điều kiện:**

```
handlebars

{{#if isAdmin}}
  <p>You are admin</p>
{{else}}
  <p>You are user</p>
{{/if}}
```

- **Lặp qua danh sách:**

```
handlebars

<ul>
  {{#each products}}
    <li>{{this.name}} - ${{this.price}}</li>
  {{/each}}
</ul>
```

- **Partial (template con):** Tái sử dụng các phần giao diện.
 - Tạo file views/partials/header.handlebars
 - Dùng trong template chính: {{> header}}
- **Layout:** Có thể định nghĩa layout chung cho toàn bộ trang.

✓ CÁC NỘI DUNG TRỌNG TÂM, CỐT LÕI CẦN LƯU Ý:

1. Hiểu rõ mục đích và vai trò của Handlebars

Điều quan trọng nhất là bạn phải nắm được **tại sao chúng ta lại cần Handlebars?** và **nó giải quyết vấn đề gì?**

- **Tách biệt logic và giao diện (Separation of Concerns):** Đây là nguyên tắc vàng. Hãy nhớ rằng Handlebars giúp bạn giữ cho mã xử lý dữ liệu và logic nghiệp vụ (trong file JavaScript) tách bạch hoàn toàn với mã hiển thị (trong file HTML/Handlebars). Điều này giúp ứng dụng của bạn dễ đọc, dễ bảo trì và dễ mở rộng hơn rất nhiều. Tưởng tượng một căn nhà mà mọi thứ đều lộn xộn, không có ngăn kéo hay tủ đựng đồ, đó là lúc bạn không có sự tách biệt!
- **Tạo ra HTML động:** Handlebars cho phép bạn "điền" dữ liệu từ máy chủ vào các mẫu HTML tĩnh, biến chúng thành các trang web động gửi đến người dùng. Nó giống như việc bạn có một bản mẫu thư mời và chỉ cần điền tên, địa chỉ, thời gian vào là xong, thay vì viết lại từng lá thư một.

2. Thành thạo cú pháp cơ bản và các khối xây dựng chính

Bạn không cần phải nhớ tất cả các cú pháp phức tạp ngay lập tức, nhưng hãy tập trung vào các thành phần cốt lõi sau:

- **Biến (Variables):** Đây là thứ bạn sẽ dùng nhiều nhất. Hãy chắc chắn bạn hiểu cách sử dụng `{{tên_biến}}` để hiển thị dữ liệu được truyền từ Express.
- **Các Helper cơ bản:**
 - **Vòng lặp `{{#each}}` ... `{{/each}}`:** Cực kỳ quan trọng để hiển thị danh sách dữ liệu (ví dụ: danh sách sản phẩm, danh sách người dùng). Hãy luyện tập thật kỹ với nó.
 - **Điều kiện `{{#if}}` ... `{{/if}}` và `{{else}}`:** Để hiển thị nội dung có điều kiện (ví dụ: hiển thị nút đăng nhập nếu người dùng chưa đăng nhập).
- **Partials (Các phần template có thể tái sử dụng):** Nắm được cách tạo và nhúng các partials (`{{> tên_partial}}`) là chìa khóa để xây dựng các template hiệu quả và tránh lặp lại mã. Hãy nghĩ đến các thành phần dùng chung như header, footer, thanh điều hướng.
- **Layouts (Bố cục):** Hiểu được vai trò của một file layout chính (thường là `main.handlebars`) chứa cấu trúc HTML chung và vị trí `{{{body}}}` để chèn nội dung của các trang cụ thể vào. Đây là xương sống cho cấu trúc giao diện của bạn.

3. Quy trình tích hợp và luồng dữ liệu với Express

Cuối cùng, việc hiểu cách Handlebars hoạt động "trong" một ứng dụng Node.js với Express là rất quan trọng:

- **Cài đặt và cấu hình `express-handlebars`:** Biết cách cài đặt thư viện và cấu hình nó trong file `app.js` của bạn để Express biết cách sử dụng Handlebars làm công cụ render view.

- **Cách Express gửi dữ liệu đến Handlebars:** Nắm vững cách bạn sử dụng `res.render('tên_template', { dữ_liệu })` trong các route handler của Express để truyền dữ liệu từ máy chủ sang template Handlebars. Dữ liệu này sẽ được Handlebars sử dụng để điền vào các biến và thực hiện các logic trong template.
- **Thư mục `views`:** Luôn nhớ rằng các file `.handlebars` của bạn sẽ nằm trong thư mục `views` (hoặc thư mục bạn đã cấu hình), và Express sẽ tìm kiếm template ở đó.

✓ ỨNG DỤNG THỰC TIỄN:

1. Xây dựng các trang web động (Dynamic Web Pages)

Đây là ứng dụng cơ bản và phổ biến nhất của Handlebars:

- **Hiển thị danh sách dữ liệu từ Database:**
 - **Thực tiễn:** Bạn có một danh sách sản phẩm, bài viết, người dùng, v.v., được lấy từ cơ sở dữ liệu (MongoDB, MySQL, PostgreSQL).
 - **Ứng dụng Handlebars:** Sử dụng vòng lặp `{{#each}}` để duyệt qua mảng dữ liệu và hiển thị từng mục một trên trang web.
 - **Ví dụ:**

JavaScript

```
// Trong Express route
app.get('/products', async (req, res) => {
  const products = await Product.find(); // Lấy dữ liệu từ DB
  res.render('products', { products: products, title: 'Danh
sách sản phẩm' });
});
```

HTML

```
<h1>{{title}}</h1>
<div class="product-list">
  {{#each products}}
    <div class="product-item">
      <h2>{{this.name}}</h2>
      <p>{{this.description}}</p>
      <span>Giá: {{this.price}} VND</span>
      <a href="/products/{{this._id}}">Xem chi tiết</a>
    </div>
  {{/each}}
</div>
```

- **Hiển thị chi tiết một đối tượng:**
 - **Thực tiễn:** Khi người dùng nhấp vào "Xem chi tiết" của một sản phẩm/bài viết.
 - **Ứng dụng Handlebars:** Lấy dữ liệu chi tiết của đối tượng đó từ DB và truyền vào template để hiển thị các thuộc tính cụ thể.
 - **Ví dụ:** `res.render('product-detail', { product: singleProduct, title: singleProduct.name });`

2. Tái sử dụng giao diện với Partial và Layouts

Đây là xương sống của việc xây dựng giao diện hiệu quả:

- **Layouts (Bố cục chính):**
 - **Thực tiễn:** Hầu hết các trang trong website của bạn đều có cấu trúc chung: Header, Footer, Thanh điều hướng (Navbar), và một vùng chứa nội dung chính.
 - **Ứng dụng Handlebars:** Định nghĩa một file layout chính (thường là `views/layouts/main.handlebars`) chứa toàn bộ cấu trúc HTML cơ bản, CSS, JavaScript chung. Vùng nội dung chính sẽ được đánh dấu bằng `{{body}}`.
 - **Lợi ích:** Không cần lặp lại mã HTML cho các phần chung trên mỗi trang, giúp dễ dàng thay đổi giao diện chung của toàn bộ website.
- **Partials (Các thành phần nhỏ có thể tái sử dụng):**
 - **Thực tiễn:** Các thành phần như thanh điều hướng, form đăng nhập nhỏ, card sản phẩm/bài viết, biểu tượng mạng xã hội.
 - **Ứng dụng Handlebars:** Tạo các file `.handlebars` riêng biệt cho từng partial (ví dụ: `views/partials/_navbar.handlebars`, `views/partials/_footer.handlebars`). Sau đó, bạn chỉ cần nhúng chúng vào bất kỳ template nào bằng `{{> partial_name}}`.
 - **Lợi ích:** Giảm sự lặp lại mã (DRY - Don't Repeat Yourself), làm cho code gọn gàng, dễ bảo trì và cập nhật. Nếu bạn thay đổi thanh điều hướng, chỉ cần sửa một file partial duy nhất.

3. Xử lý logic hiển thị với Helpers và Conditional Blocks

- **Điều kiện hiển thị (Conditional Blocks - `{{#if}}`, `{{#unless}}`):**
 - **Thực tiễn:**
 - Hiển thị nút "Đăng xuất" nếu người dùng đã đăng nhập, và nút "Đăng nhập/Đăng ký" nếu chưa.
 - Hiển thị thông báo "Không có sản phẩm nào" nếu danh sách sản phẩm trống.
 - Thay đổi trạng thái của một mục (ví dụ: "Đã hoàn thành" hoặc "Chưa hoàn thành").
 - **Ứng dụng Handlebars:**

HTML

```
{{#if userLoggedIn}}
  <a href="/logout">Đăng xuất</a>
{{else}}
  <a href="/login">Đăng nhập</a>
  <a href="/register">Đăng ký</a>
{{/if}}

{{#unless products.length}}
  <p>Hiện chưa có sản phẩm nào.</p>
{{/unless}}
```

- **Custom Helpers (Hàm trợ giúp tùy chỉnh):**
 - **Thực tiễn:**
 - Định dạng ngày tháng theo một chuẩn cụ thể (ví dụ: DD/MM/YYYY).
 - Cắt ngắn một đoạn văn bản dài và thêm "..." vào cuối.
 - So sánh hai giá trị trong template để hiển thị nội dung phù hợp.
 - **Ứng dụng Handlebars:** Bạn có thể tự định nghĩa các hàm helper trong file cấu hình Handlebars và sử dụng chúng trực tiếp trong template.
 - **Ví dụ (helper định dạng ngày):**

JavaScript

```
// Trong cấu hình express-handlebars
app.engine('handlebars', engine({
  // ...
  helpers: {
    formatDate: (dateString) => {
      const date = new Date(dateString);
      return date.toLocaleDateString('vi-VN'); // Định dạng
      theo kiểu Việt Nam
    },
    // ... các helper khác
  }
}));
```

HTML

<p>Ngày đăng: {{formatDate createdAt}}</p>

4. Xây dựng các trang Form và xử lý lỗi

- **Hiển thị dữ liệu trong Form để chỉnh sửa:**
 - **Thực tiễn:** Khi người dùng muốn chỉnh sửa thông tin cá nhân, bài viết, hoặc sản phẩm.
 - **Ứng dụng Handlebars:** Đổ dữ liệu hiện có vào các trường input của form.
 - **Ví dụ:** <input type="text" name="name" value="{{product.name}}">
- **Hiển thị thông báo lỗi hoặc thông báo thành công:**
 - **Thực tiễn:** Sau khi submit form, nếu có lỗi xác thực hoặc nếu thao tác thành công.
 - **Ứng dụng Handlebars:** Truyền các thông báo này từ Express vào template và sử dụng {{#if}} để hiển thị chúng.
 - **Ví dụ:**

JavaScript

```
// Trong Express route (ví dụ sau khi xử lý form)
if (errors.length > 0) {
  res.render('edit-product', { product: req.body, errors:
  errors });
} else {
  req.flash('success_msg', 'Sản phẩm đã được cập nhật.');
  res.redirect('/products');
}
```

HTML

```
{{#if errors}}
  <div class="alert alert-danger">
    {{#each errors}}
      <p>{{this.msg}}</p>
    {{/each}}
  </div>
{{/if}}

{{#if success_msg}}
  <div class="alert alert-success">{{success_msg}}</div>
{{/if}}
```

CHI TIẾT MỘT SỐ NỘI DUNG:

I. Handlebars là gì và tại sao nó lại quan trọng?

Handlebars là một **templating engine (công cụ tạo khuôn mẫu)** phía máy chủ (và cũng có thể phía máy khách) dựa trên JavaScript. Nó là một sự mở rộng của Mustache.js, mang lại nhiều tính năng mạnh mẽ hơn nhưng vẫn giữ được sự đơn giản trong cú pháp.

Tại sao nó lại quan trọng?

1. **Tách biệt Concerns (Separation of Concerns - SoC):** Đây là nguyên tắc thiết kế phần mềm cốt lõi mà Handlebars giúp bạn thực hiện triệt để. Thay vì trộn lẫn code JavaScript (logic nghiệp vụ) và HTML (giao diện người dùng) trong cùng một file, Handlebars cho phép bạn:
 - **Phía máy chủ (Express):** Xử lý dữ liệu, truy vấn database, định tuyến, xác thực người dùng.
 - **Phía giao diện (Handlebars):** Tập trung hoàn toàn vào việc trình bày dữ liệu mà máy chủ gửi đến. Điều này tạo ra một kiến trúc ứng dụng sạch sẽ, dễ đọc, dễ kiểm thử và dễ bảo trì hơn rất nhiều. Hãy tưởng tượng bạn đang xây một ngôi nhà, bạn sẽ muốn có các đội ngũ riêng biệt cho việc xây móng, xây tường, và trang trí nội thất, thay vì một đội ngũ làm tất cả mọi thứ cùng lúc.
2. **Khả năng tái sử dụng (Reusability):** Với các tính năng như **Partials** và **Layouts**, Handlebars khuyến khích bạn viết mã giao diện theo từng module nhỏ, có thể tái sử dụng ở nhiều nơi khác nhau. Điều này không chỉ tiết kiệm thời gian mà còn đảm bảo tính nhất quán của giao diện người dùng trên toàn bộ ứng dụng.
3. **Tăng tốc độ phát triển:** Bằng cách cung cấp một cú pháp đơn giản, trực quan để chèn dữ liệu và logic trình bày cơ bản vào HTML, Handlebars giúp lập trình viên nhanh chóng xây dựng các trang web động mà không cần phải "hardcode" mọi thứ.
4. **Dễ học và cộng đồng lớn:** Cú pháp của Handlebars khá giống với HTML, dễ đọc và dễ hiểu. Hơn nữa, nó có một cộng đồng người dùng lớn, đồng nghĩa với việc bạn có thể dễ dàng tìm thấy tài liệu, ví dụ và sự hỗ trợ khi gặp vấn đề.

II. Cấu trúc và tích hợp chuyên sâu với Node.js và Express

Để Handlebars hoạt động trơn tru với Express, bạn cần hiểu sâu hơn về quá trình cài đặt và cấu hình:

1. **Cài đặt các gói cần thiết:**
 - `npm install express-handlebars`: Đây là thư viện cầu nối giữa Express và Handlebars. Nó cung cấp các tiện ích để dễ dàng cấu hình Handlebars làm view engine cho Express.
 - `npm install handlebars`: Đây là thư viện Handlebars lõi. `express-handlebars` sẽ sử dụng nó.
2. **Cấu hình trong `app.js` (hoặc file khởi tạo Express của bạn):**

JavaScript

```
const express = require('express');
const { engine } = require('express-handlebars'); // Đúng cú pháp cho phiên bản mới hơn
const path = require('path'); // Module path để xử lý đường dẫn tuyệt đối
```

```

const app = express();

// --- CẤU HÌNH CHUYÊN SÂU HANDLEBARS ---
app.engine('handlebars', engine({
  // 1. defaultLayout: 'main' - File layout mặc định cho tất cả các
  views.
  // Mọi view khi được render mà không chỉ định layout khác sẽ
  dùng 'main'.
  // Đường dẫn sẽ là `./views/layouts/main.handlebars` theo mặc
  định.
  defaultLayout: 'main',

  // 2. layoutsDir: path.join(__dirname, 'views/layouts') - Chỉ định
  thư mục chứa layouts.
  // `path.join` giúp tạo đường dẫn tuyệt đối, tương thích trên
  mọi hệ điều hành.
  // Mặc định là `./views/layouts/`.
  layoutsDir: path.join(__dirname, 'views/layouts'),

  // 3. partialsDir: path.join(__dirname, 'views/partials') - Chỉ
  định thư mục chứa partials.
  // Mặc định là `./views/partials/`. Rất quan trọng cho việc tái
  sử dụng code.
  partialsDir: path.join(__dirname, 'views/partials'),

  // 4. extname: '.handlebars' - Chỉ định phần mở rộng của các file
  template.
  // Mặc định là '.handlebars'. Bạn có thể dùng '.hbs' nếu muốn
  ngắn gọn hơn.
  extname: '.handlebars',

  // 5. helpers: { ... } - Đối tượng chứa các hàm trợ giúp tùy chỉnh.
  // Đây là nơi bạn định nghĩa các hàm logic phức tạp hơn cho
  template.
  // Sẽ được giải thích chi tiết hơn ở phần Helpers.
  helpers: {
    // Ví dụ một helper đơn giản
    getFullYear: () => new Date().getFullYear(),
    eq: (a, b) => a === b, // Helper để so sánh hai giá trị
    // ... thêm các helper khác
  }
}));

// Thiết lập view engine cho Express là Handlebars
app.set('view engine', 'handlebars');

// Thiết lập thư mục chứa các view files (.handlebars)
// Mặc định là `./views/`.
app.set('views', path.join(__dirname, 'views'));

// --- Các cấu hình Express khác ---
app.use(express.static(path.join(__dirname, 'public'))); // Serve các
file tĩnh (CSS, JS client, hình ảnh)
app.use(express.urlencoded({ extended: true })); // Cho phép đọc dữ
liệu từ form HTML
app.use(express.json()); // Cho phép đọc dữ liệu JSON trong request
body

// --- Định nghĩa Routes ---
app.get('/', (req, res) => {
  // Render file 'home.handlebars' (trong views/)
  // Nó sẽ được chèn vào 'main.handlebars' (trong views/layouts/)

```



```

    // Dữ liệu { title: 'Trang chủ của tôi' } sẽ được truyền vào
    template
    res.render('home', { title: 'Trang chủ của tôi', isHomePage: true
    });
    });

app.get('/about', (req, res) => {
    res.render('about', { title: 'Về chúng tôi', isAboutPage: true });
});

// Khởi động server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server chạy trên cổng ${PORT}`));

```

3. Cấu trúc thư mục điển hình:

```

4. .
5. |— app.js                # File chính của ứng dụng Express
6. |— package.json
7. |— node_modules/
8. |— public/              # Chứa các file tĩnh (CSS, JS client-side,
   |  images)
9. |   |— css/
10. |   |   |— style.css
11. |   |   |— js/
12. |   |   |   |— client.js
13. |   |— views/          # Thư mục chứa tất cả các file .handlebars
14. |   |   |— layouts/    # Chứa các layout files
15. |   |   |   |— main.handlebars # Layout mặc định
16. |   |   |   |— partials/    # Chứa các partial files
17. |   |   |       |— _header.handlebars
18. |   |   |       |— _footer.handlebars
19. |   |   |       |— _navbar.handlebars
20. |   |   |— home.handlebars    # View cho trang chủ
21. |   |   |— about.handlebars   # View cho trang giới thiệu
22. |   |   |— contact.handlebars # View cho trang liên hệ

```

- o **Lưu ý đặt tên Partial:** Theo quy ước, nhiều dev đặt tên file partials với dấu gạch dưới ở đầu (ví dụ `_navbar.handlebars`) để dễ phân biệt với các view thông thường, mặc dù Handlebars không bắt buộc.

III. Các kỹ thuật Templating chuyên sâu

Đây là phần cốt lõi để bạn có thể xây dựng các giao diện phức tạp và linh hoạt.

1. Variables (Biến):

- o **Cú pháp:** `{{variableName}}`
- o **Truy cập thuộc tính lồng nhau:** `{{user.profile.name}}`
- o **Hiển thị giá trị đặc biệt:**
 - `{{this}}`: Đại diện cho ngữ cảnh hiện tại. Rất hữu ích bên trong các vòng lặp `{{#each}}`.
 - `{{@index}}`, `{{@key}}`: Trong vòng lặp `{{#each}}`, `@index` cung cấp chỉ số của phần tử trong mảng, `@key` cung cấp khóa của thuộc tính trong đối tượng.
- o **Escaping HTML:** Mặc định, Handlebars sẽ tự động thoát các ký tự HTML (`<`, `>`, `&`, `"`, `'`) để ngăn chặn các cuộc tấn công XSS (Cross-Site Scripting).
 - **Ví dụ:** Nếu `data = { description: '<h1>Hello</h1>' }`, thì `{{description}}` sẽ hiển thị `<h1>Hello</h1>`.

- **Unescaped HTML:** Nếu bạn *chắc chắn* rằng dữ liệu của mình an toàn và muốn hiển thị HTML thô, bạn sử dụng cú pháp ba dấu ngoặc nhọn: `{{{variableName}}}`.
 - **Cảnh báo:** CHỈ sử dụng khi bạn biết rõ dữ liệu đến từ đâu và đã được làm sạch (sanitized), nếu không, ứng dụng của bạn có thể dễ bị tấn công XSS.
 - **Ví dụ:** `{{{post.content}}}` (nếu `post.content` chứa HTML được lấy từ trình soạn thảo rich-text).

2. Built-in Helpers (Các hàm trợ giúp tích hợp sẵn):

- `{{#if condition}}...{{else}}...{{/if}}`:
 - **Điều kiện:** `condition` có thể là một biến boolean, một giá trị truthy/falsy, hoặc một thuộc tính của đối tượng.
 - **Truthiness/Falsiness:** Handlebars coi `false`, `undefined`, `null`, `""` (chuỗi rỗng), `0` (số 0) là falsy. Mọi thứ khác là truthy.
 - **Chuyên sâu:** Bạn có thể lồng các câu lệnh `if` vào nhau.

HTML

```
{{#if user.isAdmin}}
  <button>Quản trị</button>
  {{#if user.isActive}}
    <p>Tài khoản đang hoạt động</p>
  {{/if}}
{{else}}
  <button>Người dùng thường</button>
{{/if}}
```

- `{{#unless condition}}...{{/unless}}`:
 - Hoạt động ngược lại với `if`. Hiển thị nội dung nếu `condition` là falsy.

HTML

```
{{#unless products.length}}
  <p>Không có sản phẩm nào để hiển thị.</p>
{{/unless}}
```

- `{{#each arrayOrObject}}...{{/each}}`:
 - **Lặp qua mảng:**

HTML

```
<ul>
  {{#each users}}
    <li>{{this.name}} (Index: {{@index}})</li>
  {{else}} <li>Không có người dùng nào.</li>
  {{/each}}
</ul>
```

- **Lặp qua đối tượng (properties):**

HTML

```
<dl>
  {{#each userProfile}}
    <dt>{{@key}}:</dt> <dd>{{this}}</dd>
  {{/each}}
```

</dl>

Nếu `userProfile = { name: 'Alice', age: 30 }`, nó sẽ hiển thị `"name: Alice", "age: 30"`.

- o `{{#with context}}...{{/with}}`:
 - Thay đổi ngữ cảnh hiện tại bên trong khối. Hữu ích khi bạn muốn truy cập nhiều thuộc tính của một đối tượng mà không cần lặp lại tên đối tượng.

HTML

```
{{#with user.profile}}
  <p>Tên: {{firstName}} {{lastName}}</p>
  <p>Email: {{email}}</p>
{{/with}}
```

Tương đương với:

HTML

```
<p>Tên: {{user.profile.firstName}} {{user.profile.lastName}}</p>
<p>Email: {{user.profile.email}}</p>
```

3. Custom Helpers (Hàm trợ giúp tùy chỉnh):

- o **Tầm quan trọng:** Đây là nơi bạn mở rộng sức mạnh của Handlebars để xử lý các logic trình bày phức tạp hơn mà không làm bẩn template bằng code JavaScript thừa. Mục tiêu là giữ cho template *chỉ làm nhiệm vụ trình bày*.
- o **Ví dụ phổ biến:**
 - **Định dạng ngày tháng:** Chuyển đổi timestamp thành chuỗi ngày dễ đọc.
 - **Cắt ngắn chuỗi:** Cắt bớt đoạn văn bản dài và thêm dấu "..."
 - **Logic so sánh phức tạp:** So sánh lớn hơn, nhỏ hơn, hoặc kiểm tra nhiều điều kiện.
 - **Highlight nội dung:** Tô sáng các từ khóa tìm kiếm.
 - **Tạo URL động:** Tạo các đường dẫn tùy chỉnh dựa trên dữ liệu.
- o **Cách tạo:** Được định nghĩa trong đối tượng `helpers` khi cấu hình `express-handlebars`.
- o **Helper với tham số:**

JavaScript

```
helpers: {
  truncate: (str, len) => {
    if (str.length > len) {
      return str.substring(0, len) + '...';
    }
    return str;
  },
  // Helper kiểm tra điều kiện
  ifEquals: (arg1, arg2, options) => {
    return (arg1 == arg2) ? options.fn(this) :
    options.inverse(this);
  }
}
```

HTML

```
<p>{{truncate post.content 100}}</p>

{{#ifEquals user.role 'admin'}}
  <button>Quản lý người dùng</button>
{{else}}
  <p>Bạn không có quyền quản lý.</p>
{{/ifEquals}}
```

- **Lưu ý options trong helper:** Khi bạn tạo một helper dạng khối (`{{#ifEquals ...}}`), Handlebars sẽ truyền vào một đối tượng `options` chứa hai hàm quan trọng:
 - `options.fn(this)`: Thực thi khối code bên trong helper (`ifEquals true`).
 - `options.inverse(this)`: Thực thi khối code trong `else` (nếu có) (`ifEquals false`). `this` ở đây là ngữ cảnh (context) hiện tại của template.

4. Partials (Các phần template có thể tái sử dụng):

- **Tầm quan trọng:** Giảm trùng lặp mã (DRY - Don't Repeat Yourself).
- **Cách dùng:** `{{> partialName}}` hoặc `{{> partialName context}}` nếu muốn truyền ngữ cảnh khác vào partial.
- **Ví dụ:** Thay vì viết lại mã cho một "card sản phẩm" trên trang danh sách sản phẩm và trang kết quả tìm kiếm, bạn tạo một partial:

HTML

```
<div class="product-card">
  <h3>{{name}}</h3>
  <p>{{description}}</p>
  <span class="price">{{price}} VND</span>
  <a href="/products/{{id}}">Xem chi tiết</a>
</div>
```

HTML

```
<div class="product-grid">
  {{#each products}}
    {{> _productCard this}}
  {{/each}}
</div>
```

Bạn cũng có thể truyền dữ liệu riêng cho partial: `{{> _myPartial myData}}`

5. Layouts (Bố cục chính):

- **Tầm quan trọng:** Cung cấp cấu trúc HTML chung cho tất cả các trang web.
- **Cú pháp:** Vùng nội dung chính của từng view sẽ được chèn vào vị trí có `{{{body}}}` trong layout.
- **Truyền dữ liệu cho Layout:** Dữ liệu bạn truyền cho `res.render()` (ví dụ `res.render('home', { title: 'Trang chủ' })`) sẽ có sẵn cả trong view (`home.handlebars`) và trong layout (`main.handlebars`). Điều này cho phép bạn đặt các tiêu đề trang động (`<title>{{title}}</title>`) hoặc các yếu tố giao diện khác trong layout dựa trên dữ liệu của trang cụ thể.
- **Ví dụ views/layouts/main.handlebars:**

HTML

```
<!DOCTYPE html>
```

```

<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>{{title}} - Tên Ứng Dụng Của Tôi</title>
  <link rel="stylesheet" href="/css/style.css">
</head>
<body>
  {{> _navbar}} <main class="container">
    {{{body}}} </main>

    {{> _footer}} <script src="/js/client.js"></script>
</body>
</html>

```

- **Layout tùy chỉnh:** Bạn có thể chỉ định một layout khác cho một view cụ thể: `res.render('admin-dashboard', { layout: 'admin-layout' });` (sẽ tìm `views/layouts/admin-layout.handlebars`).

IV. Những điểm cần lưu ý và cân nhắc

- **Tính bảo mật (XSS):** Luôn nhớ rằng `{{{body}}}` và `{{{variable}}}` có thể là lỗ hổng nếu bạn không kiểm soát chặt chẽ dữ liệu. Luôn làm sạch (sanitize) dữ liệu từ người dùng trước khi hiển thị nó dưới dạng HTML không thoát. Các thư viện như `dompurify` có thể giúp ích.
- **Hiệu suất:** Đối với các ứng dụng có lượng lớn dữ liệu hoặc logic trình bày cực kỳ phức tạp, có thể có sự cân nhắc về hiệu suất. Tuy nhiên, với hầu hết các ứng dụng web thông thường, Handlebars cung cấp hiệu suất tốt.
- **Phân biệt Server-Side Rendering (SSR) và Client-Side Rendering (CSR):**
 - **SSR (Handlebars):** HTML được tạo ra hoàn chỉnh trên máy chủ và gửi đến trình duyệt. Tốt cho SEO, thời gian tải trang ban đầu nhanh.
 - **CSR (React, Vue, Angular):** Máy chủ chỉ gửi một trang HTML trống hoặc rất ít nội dung, sau đó JavaScript trên trình duyệt sẽ fetch dữ liệu và render giao diện. Thích hợp cho các ứng dụng web phức tạp, tương tác cao (Single Page Applications - SPAs).
 - Handlebars là một công cụ SSR. Bạn có thể kết hợp SSR và CSR trong cùng một ứng dụng (ví dụ: dùng Handlebars cho các trang tĩnh/ít tương tác, và dùng React cho các component phức tạp).
- **Context (Ngữ cảnh):** Hiểu rõ `this` đang đại diện cho dữ liệu nào trong từng ngữ cảnh của template (ví dụ: bên trong `each`, `this` là phần tử hiện tại của mảng).
- **Error Handling:** Nếu có lỗi trong template (ví dụ: biến không tồn tại), Handlebars sẽ trả về chuỗi rỗng cho biến đó mà không gây lỗi ứng dụng. Điều này đôi khi gây khó khăn khi debug. Hãy luôn kiểm tra dữ liệu bạn truyền vào template.

✓ TÓM TẮT CÁC BƯỚC DÙNG HANDLERBARS TRONG DỰ ÁN NODEJS

Bước 1: Khởi tạo Project NodeJS và Cài đặt các Package cần thiết

- **Mục tiêu:** Chuẩn bị môi trường phát triển.
- **Hành động:**
 1. Tạo thư mục dự án mới.
 2. Khởi tạo dự án Node.js: `npm init -y`.
 3. Cài đặt Express (framework web): `npm install express`.
 4. Cài đặt `express-handlebars` (tích hợp Handlebars cho Express) và `handlebars` (core engine): `npm install express-handlebars handlebars`.

Bước 2: Cấu hình Handlebars trong file `app.js` (hoặc file khởi tạo server)

- **Mục tiêu:** Cho Express biết cách sử dụng Handlebars làm công cụ render giao diện.
- **Hành động:**
 1. Import `express`, `express-handlebars` và `path`.
 2. Tạo instance của Express app.
 3. Sử dụng `app.engine()` để đăng ký Handlebars làm view engine, cấu hình các tùy chọn như `defaultLayout`, `layoutsDir`, `partialsDir`, `extname` và `helpers` (hàm trợ giúp tùy chỉnh).
 4. Sử dụng `app.set('view engine', 'handlebars')` và `app.set('views', 'đường_dẫn_thư_mục_views')` để thiết lập.

Bước 3: Tạo Cấu trúc thư mục cho Views (Giao diện)

- **Mục tiêu:** Tổ chức các file giao diện một cách logic và dễ quản lý.
- **Hành động:**
 1. Tạo thư mục `views` ở thư mục gốc của dự án.
 2. Bên trong `views`, tạo các thư mục con:
 - `layouts`: Chứa file layout chính (ví dụ: `main.handlebars`).
 - `partials`: Chứa các thành phần giao diện nhỏ có thể tái sử dụng (ví dụ: `_header.handlebars`, `_footer.handlebars`).
 - Các file `.handlebars` khác: Chứa nội dung chính của từng trang (ví dụ: `home.handlebars`, `about.handlebars`).

Bước 4: Tạo Layout Chính (Ví dụ: `main.handlebars`)

- **Mục tiêu:** Định nghĩa cấu trúc HTML chung cho toàn bộ website.
- **Hành động:**
 1. Tạo file `main.handlebars` trong thư mục `views/layouts`.
 2. Viết cấu trúc HTML cơ bản (`<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`).
 3. Đặt placeholder `{{body}}` vào vị trí bạn muốn nội dung của các view khác được chèn vào.
 4. Sử dụng `{{> partial_name}}` để nhúng các partial chung (ví dụ: `header`, `footer`, `navbar`).
 5. Sử dụng `{{variable_name}}` để hiển thị dữ liệu động (ví dụ: tiêu đề trang).

Bước 5: Tạo Các Partial (Tùy chọn)

- **Mục tiêu:** Xây dựng các thành phần giao diện nhỏ, tái sử dụng được.
- **Hành động:**
 1. Tạo các file `.handlebars` trong thư mục `views/partials` (ví dụ: `_navbar.handlebars`).
 2. Viết mã HTML/Handlebars cho thành phần đó.
 3. Nhúng partial vào layout hoặc các view khác bằng `{{> partial_name}}`.

Bước 6: Tạo các View Cụ thể (Ví dụ: `home.handlebars`)

- **Mục tiêu:** Xây dựng nội dung riêng của từng trang.
- **Hành động:**
 1. Tạo các file `.handlebars` trong thư mục `views` (ví dụ: `home.handlebars`, `about.handlebars`).
 2. Viết mã HTML và sử dụng cú pháp Handlebars (`{{variable}}`, `{{#each}}`, `{{#if}}`, `{{helper}}`) để hiển thị dữ liệu và logic trình bày.
 3. **Lưu ý:** Các view này chỉ chứa nội dung chính của trang, chúng sẽ được chèn vào placeholder `{{body}}` trong layout chính.

Bước 7: Định tuyến và Render View trong Express

- **Mục tiêu:** Xử lý yêu cầu HTTP và gửi trang HTML động đến trình duyệt.
- **Hành động:**
 1. Trong `app.js`, định nghĩa các route handler (`app.get()`, `app.post()`).
 2. Trong mỗi route handler, sử dụng `res.render('view_name', { data_object })` để:
 - Chỉ định tên file view cần render (ví dụ: `'home'` sẽ render `views/home.handlebars`).
 - Truyền một đối tượng JavaScript chứa dữ liệu cần hiển thị lên giao diện. Các key trong đối tượng này sẽ trở thành các biến trong Handlebars template.

Bước 8: Chạy ứng dụng NodeJS

- **Mục tiêu:** Khởi động server và kiểm tra kết quả.
- **Hành động:**
 1. Mở terminal trong thư mục gốc của dự án.
 2. Chạy lệnh: `node app.js` (hoặc `npm start` nếu bạn đã cấu hình script trong `package.json`).
 3. Mở trình duyệt và truy cập vào địa chỉ server của bạn (ví dụ: `http://localhost:3000`).

KINH NGHIỆM THỰC TIỄN, VẤN ĐỀ CẦN LƯU Ý, VÀ CÁC MẸO (TIPS)

I. Kinh nghiệm thực tiễn

1. Luôn ưu tiên Layouts và Partials:

- **Thực tiễn:** Hầu hết các trang web đều có các phần chung như header, footer, thanh điều hướng (navbar), sidebar. Đừng bao giờ viết lại HTML cho những phần này trên mỗi trang.
- **Ứng dụng:** Sử dụng **Layouts** để định nghĩa cấu trúc HTML tổng thể và **Partials** để tạo các thành phần giao diện nhỏ, tái sử dụng được (như một card sản phẩm, một mục trong menu). Điều này giúp code sạch sẽ, dễ bảo trì và đảm bảo tính nhất quán của giao diện.
- **Ví dụ:** Nếu bạn cần thay đổi một liên kết trong thanh điều hướng, chỉ cần sửa một file `_navbar.handlebars` thay vì phải mò mẫm qua hàng chục file view khác.

2. Tách biệt logic ứng dụng và logic trình bày:

- **Thực tiễn:** Dữ liệu bạn lấy từ database có thể cần được xử lý, tính toán trước khi hiển thị.
- **Ứng dụng:** Hãy xử lý mọi logic phức tạp (ví dụ: tính tổng tiền, lọc dữ liệu, kiểm tra quyền truy cập) **trong route handler của Express** (trước khi gọi `res.render()`).
- **Lưu ý:** Handlebars chỉ nên dùng để trình bày dữ liệu. Nếu bạn thấy mình viết nhiều `if` lồng nhau, hoặc các phép tính phức tạp trong file `.handlebars`, đó là dấu hiệu bạn nên chuyển logic đó về phía server (Express) hoặc tạo **custom helper** cho nó.

3. Sử dụng Custom Helpers một cách thông minh:

- **Thực tiễn:** Đôi khi bạn cần định dạng dữ liệu (ngày tháng, tiền tệ) hoặc thực hiện các so sánh đơn giản ngay trong template.
- **Ứng dụng:** Tạo các **custom helpers** cho những tác vụ này. Điều này giữ cho template của bạn gọn gàng và dễ đọc.
- **Ví dụ:** Thay vì `{{post.createdAt.toISOString().substring(0, 10)}}`, hãy tạo một helper `{{formatDate post.createdAt}}`. Hoặc thay vì `{{#ifEquals user.role 'admin'}}`, bạn sẽ cần một helper `ifEquals`.

II. Vấn đề cần lưu ý

1. Bảo mật (Cross-Site Scripting - XSS):

- **Vấn đề:** Đây là rủi ro bảo mật lớn nhất khi hiển thị dữ liệu động. Kẻ tấn công có thể chèn mã JavaScript độc hại vào dữ liệu của bạn, và nếu bạn hiển thị nó mà không thoát (escape) đúng cách, mã đó sẽ chạy trên trình duyệt của người dùng.
- **Lưu ý:**
 - **Handlebars mặc định thoát HTML:** Đây là một tính năng bảo mật tuyệt vời. Luôn sử dụng cú pháp `{{variable}}` trừ khi bạn *chắc chắn tuyệt đối* về nguồn gốc và tính an toàn của dữ liệu.
 - **Cẩn trọng với `{{{raw_html_variable}}}`:** Chỉ sử dụng cú pháp ba dấu ngoặc nhọn khi bạn biết rõ dữ liệu đó (ví dụ: HTML từ trình soạn thảo rich-text của người dùng đáng tin cậy) và đã được **làm sạch (sanitized)** ở phía server bằng các thư viện chuyên dụng (như `DOMPurify` trên server hoặc các giải pháp bảo mật khác). Tuyệt đối không bao giờ hiển thị trực tiếp dữ liệu đầu vào từ người dùng bằng `{{{}}}`.

2. Hiểu về Context (Ngữ cảnh):

- **Vấn đề:** Dễ nhầm lẫn `this` hoặc không truy cập được biến trong các khối (block helper) như `{{#each}}` hay `{{#with}}`.
- **Lưu ý:**
 - Trong các vòng lặp `{{#each}}`, `{{this}}` đại diện cho phần tử hiện tại trong mảng.
 - Trong các helper `{{#with context}}`, ngữ cảnh bên trong khối đã thay đổi sang `context`.
 - Nếu bạn cần truy cập biến từ ngữ cảnh cha trong một khối con, hãy truyền nó như một tham số cho helper hoặc sử dụng cú pháp truy cập toàn cục nếu helper đó hỗ trợ. Ví dụ: `{{#each users}}{{../parentVar}}{{this.name}}{{/each}}`.

3. Debug và Xử lý lỗi trong Template:

- **Vấn đề:** Khi có lỗi trong template (ví dụ: biến không tồn tại, cú pháp sai), Handlebars thường trả về một chuỗi rỗng hoặc không hiển thị gì mà không báo lỗi rõ ràng. Điều này có thể khiến việc debug trở nên khó khăn.
- **Lưu ý:**
 - **Sử dụng `{{log variable}}`:** Đây là một helper tích hợp sẵn của Handlebars. Nó sẽ ghi giá trị của `variable` vào console của server (nơi bạn chạy Node.js), rất hữu ích để kiểm tra dữ liệu đã truyền đến template hay chưa và giá trị của nó là gì.
 - **Kiểm tra dữ liệu truyền vào `res.render()`:** Luôn đảm bảo rằng đối tượng dữ liệu bạn truyền vào `res.render()` chứa tất cả các biến mà template của bạn mong đợi.

4. Phân biệt SSR và CSR:

- **Vấn đề:** Đôi khi người mới học nhầm lẫn vai trò của Handlebars (Server-Side Rendering - SSR) với các framework Frontend (Client-Side Rendering - CSR) như React, Vue, Angular.
- **Lưu ý:**
 - **Handlebars là SSR:** Trang web được render hoàn toàn trên máy chủ và gửi HTML cuối cùng đến trình duyệt. Tốt cho SEO và thời gian tải ban đầu.
 - **CSR:** HTML rỗng được gửi, sau đó JavaScript tải dữ liệu và render giao diện trên trình duyệt. Tốt cho các ứng dụng phức tạp, tương tác cao (SPAs).
 - Bạn có thể kết hợp cả hai: Sử dụng Handlebars cho các trang tĩnh hoặc ít tương tác, và dùng một framework JS cho các thành phần phức tạp, tương tác cao trên trang đó (ví dụ: một biểu đồ động, một bảng dữ liệu có thể lọc/sắp xếp).

III. Các mẹo (Tips)

1. Giữ tên file partials có dấu gạch dưới `_`:

- **Mẹo:** Đặt tên file partials bắt đầu bằng dấu gạch dưới (ví dụ: `_navbar.handlebars`, `_product_card.handlebars`). Đây là một quy ước phổ biến giúp bạn dễ dàng phân biệt đâu là partial và đâu là view chính khi nhìn vào cấu trúc thư mục.

2. Sử dụng `path.join()` cho các đường dẫn:

- **Mẹo:** Khi cấu hình `layoutsDir` và `partialsDir` trong `app.js`, luôn sử dụng `path.join(__dirname, 'views/layouts')` thay vì nối chuỗi thủ công `('views/layouts').path.join()` đảm bảo đường dẫn hoạt động chính xác trên mọi hệ điều hành (Windows dùng `\` còn Linux/macOS dùng `/`).

3. Tối ưu hóa các Helper:

- **Mẹo:**
 - **Tránh logic nghiệp vụ phức tạp:** Helper nên thực hiện các tác vụ trình bày đơn giản. Nếu logic quá phức tạp, hãy đưa nó vào service layer hoặc controller của Express.
 - **Sử dụng `this` cho ngữ cảnh hiện tại:** Bên trong helper, `this` sẽ tham chiếu đến ngữ cảnh dữ liệu hiện tại mà helper được gọi.
 - **Tạo helper chung:** Viết các helper thật linh hoạt để có thể tái sử dụng ở nhiều nơi (ví dụ: một helper `formatCurrency` có thể nhận vào cả giá trị và loại tiền tệ).

4. Flash Messages cho thông báo:

- **Mẹo:** Để hiển thị các thông báo ngắn gọn (ví dụ: "Đăng nhập thành công", "Sản phẩm đã được xóa"), hãy sử dụng thư viện `connect-flash` (kết hợp với `express-session`). Bạn có thể lưu thông báo vào session, sau đó truy cập trong layout/view và hiển thị.
- **Cách dùng:**

JavaScript

```
// Trong route
req.flash('success_msg', 'Bạn đã đăng nhập thành công!');
res.redirect('/dashboard');

// Trong layout/partial để hiển thị
{{#if success_msg}}
    <div class="alert alert-success">{{success_msg}}</div>
{{/if}}
```

5. Sử dụng biến `process.env.NODE_ENV`:

- **Mẹo:** Điều này không trực tiếp liên quan đến Handlebars nhưng rất hữu ích. Bạn có thể sử dụng biến môi trường này để hiển thị các phần tử khác nhau trong giao diện tùy thuộc vào môi trường (phát triển hay sản phẩm).
- **Ví dụ:**

HTML

```
{{#ifEquals process.env.NODE_ENV 'development'}}
    <p style="color: red;">Bạn đang ở chế độ phát triển.</p>
{{/ifEquals}}
```

Bạn cần truyền `process.env.NODE_ENV` vào `res.render()` hoặc tạo một helper để truy cập nó.

EXERCISE

DỰ ÁN: HỆ THỐNG QUẢN LÝ NHIỆM VỤ ĐƠN GIẢN

I. Tóm Tắt Thông Tin Dự Án và Mục Tiêu

- **Tóm tắt thông tin:** Một ứng dụng web đơn giản cho phép người dùng thêm, xem, cập nhật và xóa các nhiệm vụ. Mỗi nhiệm vụ sẽ có tên, mô tả và trạng thái (chưa hoàn thành / đã hoàn thành).
- **Mục tiêu:**
 - Củng cố kiến thức về Node.js và Express (định tuyến, xử lý request/response).
 - Nắm vững cách tích hợp và sử dụng Handlebars (layouts, partials, helpers, biến, các khối logic) để tạo giao diện động.
 - Thực hành các thao tác CRUD cơ bản.
 - Hiểu cách truyền dữ liệu từ server (Express) sang client (Handlebars).
 - Học cách quản lý dữ liệu đơn giản mà không cần database phức tạp (sử dụng mảng trong bộ nhớ).

II. Các Chức Năng Chính

1. **Trang chủ:** Hiển thị danh sách tất cả các nhiệm vụ.
2. **Xem chi tiết nhiệm vụ:** Xem thông tin chi tiết của một nhiệm vụ cụ thể.
3. **Thêm nhiệm vụ mới:** Form để tạo nhiệm vụ với tên và mô tả.
4. **Cập nhật nhiệm vụ:** Form để chỉnh sửa thông tin và trạng thái của nhiệm vụ.
5. **Xóa nhiệm vụ:** Nút để xóa một nhiệm vụ khỏi danh sách.
6. **Thay đổi trạng thái:** Nút nhanh để chuyển đổi trạng thái "Hoàn thành" / "Chưa hoàn thành".

III. Công Nghệ Sử Dụng

- **Backend:** Node.js, Express.js
- **Templating Engine:** Handlebars (với `express-handlebars`)
- **Quản lý dữ liệu:** Mảng JavaScript trong bộ nhớ (để đơn giản hóa, không dùng database thật)
- **Styling (Tùy chọn):** Một chút CSS cơ bản để giao diện dễ nhìn hơn.

IV. Cấu Trúc Thư Mục Dự Án

```
simple-task-manager/
├── app.js                # File chính của ứng dụng Express
├── package.json
├── package-lock.json
├── node_modules/
├── public/               # Chứa các file tĩnh (CSS, JS phía client)
│   └── css/
│       └── style.css
├── views/                # Thư mục chứa tất cả các file .handlebars
│   ├── layouts/
│   │   └── main.handlebars # Layout chính của ứng dụng
│   └── partials/
│       ├── _header.handlebars # Partial cho tiêu đề (header)
│       ├── _footer.handlebars # Partial cho chân trang (footer)
│       └── _taskItem.handlebars # Partial cho một mục nhiệm vụ trong danh
sách
├── index.handlebars      # Trang chủ hiển thị danh sách nhiệm vụ
├── addTask.handlebars    # Trang form thêm nhiệm vụ
├── editTask.handlebars   # Trang form chỉnh sửa nhiệm vụ
└── taskDetail.handlebars # Trang hiển thị chi tiết nhiệm vụ
```

V. Tóm Tắt Danh Sách Các Bước Thực Hiện

1. **Thiết lập dự án cơ bản:** Khởi tạo Node.js, cài Express và Handlebars.
2. **Cấu hình Express và Handlebars:** Trong `app.js`.
3. **Tạo Cấu trúc thư mục View và file Layout chính.**
4. **Tạo CSS cơ bản (tùy chọn).**
5. **Thiết lập Dữ liệu Giả định (Mock Data).**
6. **Xây dựng Trang Chủ (Read All Tasks):**
 - o Route GET `/`.
 - o View `index.handlebars` hiển thị danh sách nhiệm vụ.
 - o `Partial _taskItem.handlebars` để hiển thị một nhiệm vụ.
7. **Xây dựng Chức năng Thêm Nhiệm vụ (Create Task):**
 - o Route GET `/tasks/add` (hiển thị form).
 - o View `addTask.handlebars` (form thêm).
 - o Route POST `/tasks` (xử lý dữ liệu form).
8. **Xây dựng Chức năng Xem Chi tiết Nhiệm vụ (Read Single Task):**
 - o Route GET `/tasks/:id`.
 - o View `taskDetail.handlebars`.
9. **Xây dựng Chức năng Chỉnh sửa Nhiệm vụ (Update Task):**
 - o Route GET `/tasks/edit/:id` (hiển thị form với dữ liệu có sẵn).
 - o View `editTask.handlebars` (form chỉnh sửa).
 - o Route POST `/tasks/edit/:id` (xử lý cập nhật).
10. **Xây dựng Chức năng Xóa Nhiệm vụ (Delete Task):**
 - o Route POST `/tasks/delete/:id`.
11. **Xây dựng Chức năng Đổi Trạng thái (Toggle Status).**
12. **Thêm Custom Helpers (Tùy chọn nhưng hữu ích).**
13. **Kiểm tra và Debug.**

VI. Chi Tiết Các Bước Thực Hiện

Bước 1: Thiết lập dự án cơ bản

1. Mở terminal:

Bash

```
mkdir simple-task-manager
cd simple-task-manager
npm init -y
npm install express express-handlebars handlebars uuid
```

(Thêm `uuid` để tạo ID duy nhất cho nhiệm vụ mà không cần DB).

Bước 2: Cấu hình Express và Handlebars trong `app.js`

Tạo file `app.js` và thêm mã sau:

JavaScript

```
const express = require('express');
const { engine } = require('express-handlebars');
const path = require('path');
const { v4: uuidv4 } = require('uuid'); // Import uuid để tạo ID
```

```

const app = express();
const PORT = 3000;

// --- Mock Data (Dữ liệu giả định trong bộ nhớ) ---
let tasks = [
  { id: uuidv4(), name: 'Học Node.js', description: 'Hoàn thành dự án quản lý nhiệm vụ', completed: false },
  { id: uuidv4(), name: 'Làm bài tập Handlebars', description: 'Nắm vững các cú pháp và helper', completed: true },
  { id: uuidv4(), name: 'Chuẩn bị bài thuyết trình', description: 'Tóm tắt các kiến thức đã học', completed: false }
];

// --- Cấu hình Handlebars ---
app.engine('handlebars', engine({
  defaultLayout: 'main',
  layoutsDir: path.join(__dirname, 'views/layouts'),
  partialsDir: path.join(__dirname, 'views/partials'),
  extname: '.handlebars',
  helpers: {
    // Custom Helpers (sẽ thêm sau)
    isCompleted: (status) => status ? 'Đã hoàn thành' : 'Chưa hoàn thành',
    ifEquals: (arg1, arg2, options) => { // Helper so sánh
      return (arg1 == arg2) ? options.fn(this) : options.inverse(this);
    }
  }
}));
app.set('view engine', 'handlebars');
app.set('views', path.join(__dirname, 'views'));

// --- Middleware ---
app.use(express.static(path.join(__dirname, 'public'))); // Phục vụ file tĩnh
app.use(express.urlencoded({ extended: true })); // Xử lý dữ liệu form (x-www-form-urlencoded)
app.use(express.json()); // Xử lý dữ liệu JSON (nếu có)

// --- Routes (sẽ thêm sau) ---

// --- Khởi động Server ---
app.listen(PORT, () => {
  console.log(`Server đang chạy tại http://localhost:${PORT}`);
});

```

Bước 3: Tạo Cấu trúc thư mục View và file Layout chính

Tạo các thư mục và file theo cấu trúc đã định nghĩa: views/, views/layouts/, views/partials/, public/, public/css/

Tạo views/layouts/main.handlebars:

HTML

```

<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{{title}} - Quản lý Nhiệm vụ</title>
  <link rel="stylesheet" href="/css/style.css">
</head>

```

```

<body>
  <header class="app-header">
    <h1>{{title}}</h1>
    <nav>
      <a href="/">Trang chủ</a> |
      <a href="/tasks/add">Thêm nhiệm vụ</a>
    </nav>
  </header>

  <main class="container">
    {{{body}}} </main>

  <footer class="app-footer">
    <p>&copy; {{getCurrentYear}} Simple Task Manager</p>
  </footer>
</body>
</html>

```

Lưu ý: {{getCurrentYear}} là một helper mà chúng ta đã định nghĩa trong app.js.

Bước 4: Tạo CSS cơ bản

Tạo file public/css/style.css:

CSS

```

body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0;
  background-color: #f4f4f4;
  color: #333;
}

.container {
  max-width: 960px;
  margin: 20px auto;
  padding: 20px;
  background-color: #fff;
  border-radius: 8px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.app-header {
  background-color: #4CAF50;
  color: white;
  padding: 15px 20px;
  text-align: center;
  border-bottom: 5px solid #45a049;
}

.app-header h1 {
  margin: 0;
}

.app-header nav a {
  color: white;
  text-decoration: none;
  margin: 0 10px;
}

.app-header nav a:hover {

```

```

        text-decoration: underline;
    }

    .app-footer {
        text-align: center;
        padding: 20px;
        margin-top: 30px;
        color: #777;
        border-top: 1px solid #eee;
    }

    .task-list {
        list-style: none;
        padding: 0;
    }

    .task-item {
        background-color: #e9e9e9;
        margin-bottom: 10px;
        padding: 15px;
        border-radius: 5px;
        display: flex;
        justify-content: space-between;
        align-items: center;
    }

    .task-item.completed {
        background-color: #d4edda; /* Màu xanh nhẹ cho nhiệm vụ đã hoàn thành */
        border-left: 5px solid #28a745;
    }

    .task-item-content {
        flex-grow: 1;
    }

    .task-item h3 {
        margin: 0 0 5px 0;
        color: #0056b3;
    }

    .task-item p {
        margin: 0;
        font-size: 0.9em;
        color: #555;
    }

    .task-actions {
        display: flex;
        gap: 10px;
        margin-left: 20px;
    }

    .task-actions a, .task-actions button, form button {
        background-color: #007bff;
        color: white;
        border: none;
        padding: 8px 12px;
        border-radius: 5px;
        cursor: pointer;
        text-decoration: none;
        font-size: 0.9em;
    }

```

```

.task-actions a:hover, .task-actions button:hover, form button:hover {
    opacity: 0.9;
}

.task-actions .edit-btn { background-color: #ffcl07; }
.task-actions .edit-btn:hover { background-color: #e0a800; }
.task-actions .delete-btn { background-color: #dc3545; }
.task-actions .delete-btn:hover { background-color: #c82333; }
.task-actions .toggle-btn { background-color: #6c757d; }
.task-actions .toggle-btn:hover { background-color: #5a6268; }

form {
    background-color: #f9f9f9;
    padding: 20px;
    border-radius: 8px;
    box-shadow: inset 0 1px 3px rgba(0,0,0,0.06);
}

form div {
    margin-bottom: 15px;
}

form label {
    display: block;
    margin-bottom: 5px;
    font-weight: bold;
}

form input[type="text"],
form textarea {
    width: 100%;
    padding: 10px;
    border: 1px solid #ccc;
    border-radius: 4px;
    box-sizing: border-box; /* Đảm bảo padding không làm tăng chiều rộng */
}

form input[type="checkbox"] {
    margin-right: 10px;
}

form button[type="submit"] {
    background-color: #28a745;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-size: 1em;
}

form button[type="submit"]:hover {
    background-color: #218838;
}

.task-detail-card {
    background-color: #e0f2f7;
    padding: 25px;
    border-radius: 8px;
    border-left: 5px solid #007bff;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.task-detail-card h2 {

```



```

        color: #0056b3;
        margin-top: 0;
    }
    .task-detail-card p {
        margin-bottom: 10px;
        line-height: 1.6;
    }
    .task-detail-card strong {
        color: #444;
    }
}

```

Bước 6: Xây dựng Trang Chủ (Read All Tasks)

Tạo views/index.handlebars:

HTML

```

<ul class="task-list">
    {{#each tasks}}
        {{> _taskItem this}} {{else}}
            <p>Chưa có nhiệm vụ nào. Hãy thêm một nhiệm vụ mới!</p>
        {{/each}}
</ul>

```

Tạo views/partials/_taskItem.handlebars:

HTML

```

<li class="task-item {{#if completed}}completed{{/if}}">
    <div class="task-item-content">
        <h3>
            <a href="/tasks/{{id}}">{{name}}</a>
        </h3>
        <p>{{description}}</p>
        <small>Trạng thái: {{isCompleted completed}}</small>
    </div>
    <div class="task-actions">
        <a href="/tasks/edit/{{id}}" class="edit-btn">Sửa</a>
        <form action="/tasks/toggle/{{id}}" method="POST"
style="display:inline;">
            <button type="submit" class="toggle-btn">{{#if completed}}Chưa
hoàn thành{{else}}Hoàn thành{{/if}}</button>
        </form>
        <form action="/tasks/delete/{{id}}" method="POST"
style="display:inline;">
            <button type="submit" class="delete-btn">Xóa</button>
        </form>
    </div>
</li>

```

Thêm route vào app.js (trước app.listen):

JavaScript

```

// Route Trang Chủ: Hiển thị tất cả nhiệm vụ
app.get('/', (req, res) => {
    res.render('index', { title: 'Tất cả nhiệm vụ', tasks: tasks });
});

```

Bước 7: Xây dựng Chức năng Thêm Nhiệm vụ (Create Task)

Tạo views/addTask.handlebars:

HTML

```
<h2>Thêm nhiệm vụ mới</h2>
<form action="/tasks" method="POST">
  <div>
    <label for="name">Tên nhiệm vụ:</label>
    <input type="text" id="name" name="name" required>
  </div>
  <div>
    <label for="description">Mô tả:</label>
    <textarea id="description" name="description" rows="4"></textarea>
  </div>
  <div>
    <button type="submit">Thêm nhiệm vụ</button>
  </div>
</form>
```

Thêm route vào app.js:

JavaScript

```
// Route hiển thị form thêm nhiệm vụ
app.get('/tasks/add', (req, res) => {
  res.render('addTask', { title: 'Thêm Nhiệm vụ Mới' });
});

// Route xử lý thêm nhiệm vụ mới
app.post('/tasks', (req, res) => {
  const { name, description } = req.body;
  if (name && description) {
    const newTask = {
      id: uuidv4(),
      name,
      description,
      completed: false
    };
    tasks.push(newTask);
    res.redirect('/'); // Redirect về trang chủ sau khi thêm
  } else {
    res.status(400).send('Tên và mô tả nhiệm vụ không được để trống.');
```

Bước 8: Xây dựng Chức năng Xem Chi tiết Nhiệm vụ (Read Single Task)

Tạo views/taskDetail.handlebars:

HTML

```
<div class="task-detail-card">
  <h2>Chi tiết nhiệm vụ: {{task.name}}</h2>
  <p><strong>Mô tả:</strong> {{task.description}}</p>
  <p><strong>Trạng thái:</strong> {{isCompleted task.completed}}</p>
  <hr>
  <a href="/tasks/edit/{{task.id}}" class="edit-btn">Sửa nhiệm vụ này</a>
  <form action="/tasks/delete/{{task.id}}" method="POST"
  style="display:inline; margin-left:10px;">
    <button type="submit" class="delete-btn">Xóa nhiệm vụ này</button>
  </form>
```

```
    <a href="/" style="margin-left:10px;">Quay lại danh sách</a>
  </div>
```

Thêm route vào app.js:

JavaScript

```
// Route xem chi tiết nhiệm vụ
app.get('/tasks/:id', (req, res) => {
  const task = tasks.find(t => t.id === req.params.id);
  if (task) {
    res.render('taskDetail', { title: `Chi tiết: ${task.name}`, task:
task });
  } else {
    res.status(404).send('Không tìm thấy nhiệm vụ.');
  }
});
```

Bước 9: Xây dựng Chức năng Chỉnh sửa Nhiệm vụ (Update Task)

Tạo views/editTask.handlebars:

HTML

```
<h2>Sửa nhiệm vụ: {{task.name}}</h2>
<form action="/tasks/edit/{{task.id}}" method="POST">
  <div>
    <label for="name">Tên nhiệm vụ:</label>
    <input type="text" id="name" name="name" value="{{task.name}}"
required>
  </div>
  <div>
    <label for="description">Mô tả:</label>
    <textarea id="description" name="description"
rows="4">{{task.description}}</textarea>
  </div>
  <div>
    <label for="completed">
      <input type="checkbox" id="completed" name="completed" {{#if
task.completed}}checked{{/if}}>
      Đã hoàn thành
    </label>
  </div>
  <div>
    <button type="submit">Cập nhật nhiệm vụ</button>
  </div>
</form>
```

Thêm route vào app.js:

JavaScript

```
// Route hiển thị form chỉnh sửa nhiệm vụ
app.get('/tasks/edit/:id', (req, res) => {
  const task = tasks.find(t => t.id === req.params.id);
  if (task) {
    res.render('editTask', { title: `Sửa: ${task.name}`, task: task });
  } else {
    res.status(404).send('Không tìm thấy nhiệm vụ để sửa.');
```

```
// Route xử lý cập nhật nhiệm vụ
app.post('/tasks/edit/:id', (req, res) => {
  const taskId = req.params.id;
  const { name, description, completed } = req.body;
  const taskIndex = tasks.findIndex(t => t.id === taskId);

  if (taskIndex !== -1) {
    tasks[taskIndex] = {
      ...tasks[taskIndex], // Giữ lại ID cũ và các thuộc tính không đổi
      name: name,
      description: description,
      completed: completed === 'on' ? true : false // Checkbox trả về
'on' nếu được chọn
    };
    res.redirect(`/tasks/${taskId}`); // Redirect về trang chi tiết sau
    khi cập nhật
  } else {
    res.status(404).send('Không tìm thấy nhiệm vụ để cập nhật.');
```

Bước 10: Xây dựng Chức năng Xóa Nhiệm vụ (Delete Task)

Thêm route vào app.js:

JavaScript

```
// Route xử lý xóa nhiệm vụ
app.post('/tasks/delete/:id', (req, res) => {
  const taskId = req.params.id;
  const initialLength = tasks.length;
  tasks = tasks.filter(t => t.id !== taskId);
  if (tasks.length < initialLength) {
    res.redirect('/'); // Redirect về trang chủ nếu xóa thành công
  } else {
    res.status(404).send('Không tìm thấy nhiệm vụ để xóa.');
```

Bước 11: Xây dựng Chức năng Đổi Trạng thái (Toggle Status)

Thêm route vào app.js:

JavaScript

```
// Route xử lý chuyển đổi trạng thái nhiệm vụ (hoàn thành/chưa hoàn thành)
app.post('/tasks/toggle/:id', (req, res) => {
  const taskId = req.params.id;
  const taskIndex = tasks.findIndex(t => t.id === taskId);

  if (taskIndex !== -1) {
    tasks[taskIndex].completed = !tasks[taskIndex].completed; // Đảo
    ngược trạng thái
    res.redirect('/'); // Quay lại trang chủ
  } else {
    res.status(404).send('Không tìm thấy nhiệm vụ để chuyển đổi trạng
    thái.');
```

Bước 12: Thêm Custom Helpers (Đã làm ở Bước 2)

Chúng ta đã định nghĩa `isCompleted` và `ifEquals` trong `app.js`.

- `isCompleted: {{isCompleted completed}}` để hiển thị "Đã hoàn thành" hoặc "Chưa hoàn thành".
- `ifEquals: {{#ifEquals task.status 'pending'}}...{{/ifEquals}}` để so sánh các giá trị phức tạp hơn.
- `getCurrentYear: {{getCurrentYear}}` để hiển thị năm hiện tại trong footer.

VII. Checklist Dự Án

- [x] Khởi tạo dự án Node.js.
- [x] Cài đặt `express`, `express-handlebars`, `handlebars`, `uuid`.
- [x] Cấu hình Express và Handlebars trong `app.js`.
- [x] Thiết lập `defaultLayout`, `layoutsDir`, `partialsDir`, `extname`.
- [x] Tạo các thư mục `views`, `views/layouts`, `views/partials`, `public/css`.
- [x] Tạo `main.handlebars` với `{{body}}` và nhúng `partials` nếu có.
- [x] Tạo `style.css` và liên kết trong `main.handlebars`.
- [x] Định nghĩa dữ liệu giả định (mock data) cho nhiệm vụ.
- [x] Viết Route GET / để hiển thị danh sách nhiệm vụ.
- [x] Tạo `index.handlebars` và sử dụng `{{#each}}`.
- [x] Tạo `_taskItem.handlebars` partial.
- [x] Viết Route GET /tasks/add để hiển thị form thêm.
- [x] Tạo `addTask.handlebars` với form POST.
- [x] Viết Route POST /tasks để xử lý thêm nhiệm vụ.
- [x] Viết Route GET /tasks/:id để xem chi tiết nhiệm vụ.
- [x] Tạo `taskDetail.handlebars`.
- [x] Viết Route GET /tasks/edit/:id để hiển thị form sửa.
- [x] Tạo `editTask.handlebars` với dữ liệu được điền sẵn.
- [x] Viết Route POST /tasks/edit/:id để xử lý cập nhật.
- [x] Viết Route POST /tasks/delete/:id để xóa nhiệm vụ.
- [x] Viết Route POST /tasks/toggle/:id để chuyển đổi trạng thái.
- [x] Đã thêm các Custom Helpers `isCompleted`, `ifEquals`, `getCurrentYear`.
- [x] Đã kiểm tra các chức năng cơ bản.

VIII. 05 Test Case

Để kiểm tra các chức năng chính của ứng dụng:

1. **Test Case 1: Thêm nhiệm vụ mới thành công.**
 - **Các bước:** Truy cập `/tasks/add`. Nhập "Mua sữa" vào tên, "Sữa tươi không đường" vào mô tả. Nhấn "Thêm nhiệm vụ".
 - **Kết quả mong đợi:** Trang chủ / hiển thị, và nhiệm vụ "Mua sữa" xuất hiện trong danh sách.
2. **Test Case 2: Chỉnh sửa nhiệm vụ thành công.**
 - **Các bước:** Trên trang chủ, tìm một nhiệm vụ đã có (ví dụ: "Học Node.js"). Nhấn "Sửa". Thay đổi tên thành "Học Node.js nâng cao" và tích vào "Đã hoàn thành". Nhấn "Cập nhật nhiệm vụ".
 - **Kết quả mong đợi:** Trang chi tiết của nhiệm vụ đó hiển thị với tên mới và trạng thái "Đã hoàn thành". Quay lại trang chủ, nhiệm vụ đó cũng đã cập nhật.
3. **Test Case 3: Chuyển đổi trạng thái nhiệm vụ.**

- **Các bước:** Trên trang chủ, tìm một nhiệm vụ "Chưa hoàn thành" (ví dụ: "Chuẩn bị bài thuyết trình"). Nhấn nút "Hoàn thành" bên cạnh nó.
 - **Kết quả mong đợi:** Nhiệm vụ "Chuẩn bị bài thuyết trình" giờ đây hiển thị là "Đã hoàn thành" và có màu nền xanh lá (nếu CSS được áp dụng đúng). Lặp lại để chuyển về "Chưa hoàn thành".
4. **Test Case 4: Xem chi tiết nhiệm vụ.**
- **Các bước:** Trên trang chủ, nhấp vào tên của một nhiệm vụ bất kỳ (ví dụ: "Làm bài tập Handlebars").
 - **Kết quả mong đợi:** Trang chi tiết nhiệm vụ hiển thị đầy đủ tên, mô tả và trạng thái của nhiệm vụ đó.
5. **Test Case 5: Xóa nhiệm vụ.**
- **Các bước:** Trên trang chủ, tìm một nhiệm vụ bất kỳ. Nhấn nút "Xóa".
 - **Kết quả mong đợi:** Nhiệm vụ đó biến mất khỏi danh sách trên trang chủ.

IX. Các Lưu Ý Lỗi và Mẹo Xử Lý Cần Thiết

- Lỗi phổ biến: Cannot find module 'express-handlebars' hoặc tương tự.**
 - **Nguyên nhân:** Chưa cài đặt đúng gói, hoặc chạy lệnh `node app.js` từ một thư mục khác không phải thư mục gốc dự án.
 - **Cách khắc phục:** Đảm bảo bạn đã chạy `npm install express-handlebars handlebars` trong thư mục gốc. Chắc chắn bạn đang ở đúng thư mục khi chạy `node app.js`.
- Lỗi: Template không được render, trang trắng hoặc lỗi 500.**
 - **Nguyên nhân:**
 - Sai đường dẫn `viewsDir`, `layoutsDir`, `partialsDir` trong cấu hình `app.engine()`.
 - Tên file `.handlebars` không khớp với tên bạn truyền vào `res.render()`.
 - Thiếu `{{body}}` trong file layout chính.
 - Sai cú pháp Handlebars trong template (ví dụ: thiếu dấu `}` hoặc `#`).
 - **Mẹo xử lý:**
 - Kiểm tra kỹ các đường dẫn trong `app.js` bằng `path.join(__dirname, '...')`.
 - Sử dụng `console.log()` trong route để kiểm tra dữ liệu bạn đang truyền đi.
 - Sử dụng `{{log variable}}` trong template để in giá trị biến ra console của server, giúp debug xem dữ liệu có đến được template không.
- Lỗi: Cannot POST /tasks/add hoặc method không đúng.**
 - **Nguyên nhân:** Bạn đang cố gắng gửi form POST đến một route chỉ định cho GET, hoặc ngược lại.
 - **Cách khắc phục:** Kiểm tra lại `method="POST"` trong tag `<form>` và đảm bảo bạn có `app.post('/your-route', ...)` tương ứng trong `app.js`. Đảm bảo đã có `app.use(express.urlencoded({ extended: true }));` để Express có thể đọc dữ liệu từ form.
- Dữ liệu không hiển thị trên template hoặc sai giá trị.**
 - **Nguyên nhân:** Tên biến trong template không khớp với key trong đối tượng dữ liệu bạn truyền từ `res.render()`.
 - **Mẹo xử lý:** Dùng `console.log(data)` trong route ngay trước `res.render('view', data)`; để xem dữ liệu bạn đang gửi đi có đúng cấu trúc không. Dùng `{{log variable}}` trong template để xem Handlebars nhận được giá trị gì.
- Vấn đề về cache (khi thay đổi code mà không thấy cập nhật).**
 - **Mẹo:** Đối với phát triển, hãy sử dụng `nodemon` (`npm install -g nodemon`). Sau đó, bạn có thể chạy `nodemon app.js` thay vì `node app.js`. `nodemon` sẽ tự động khởi động lại server mỗi khi bạn lưu thay đổi trong file code, giúp bạn thấy ngay kết quả.
- Quản lý ID (khi không dùng database):**

- **Mẹo:** Để đảm bảo ID duy nhất cho mỗi nhiệm vụ khi dữ liệu chỉ nằm trong bộ nhớ, hãy sử dụng thư viện `uuid` như đã làm trong ví dụ. Điều này giúp bạn dễ dàng tìm kiếm, cập nhật và xóa các mục một cách chính xác.

7. Sử dụng Global/Context Data trong Handlebars:

- **Mẹo:** Nếu bạn có một biến nào đó cần xuất hiện trên tất cả các trang (ví dụ: thông tin người dùng đăng nhập, tên ứng dụng), bạn có thể truyền nó vào `res.render()` cho mỗi route, hoặc tạo một middleware để inject dữ liệu đó vào `res.locals`, sau đó nó sẽ có sẵn trong tất cả các template.

JavaScript

```
// Ví dụ middleware để inject data
app.use((req, res, next) => {
  res.locals.appName = 'Simple Task Manager';
  next();
});

// Trong template: {{appName}}
```

EXERCISE 2: ỨNG DỤNG QUẢN LÝ BLOG ĐƠN GIẢN

I. Tóm Tắt Mục Tiêu

Mục tiêu chính của bài tập này là giúp bạn:

- **Kết nối và tương tác với MongoDB:** Học cách thiết lập Mongoose, định nghĩa Schema/Model và thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên cơ sở dữ liệu MongoDB.
- **Vận dụng sâu Handlebars:** Sử dụng các tính năng như layouts, partials, helpers để hiển thị dữ liệu từ database một cách động và có cấu trúc.
- **Xử lý form và dữ liệu người dùng:** Nắm vững việc lấy dữ liệu từ form HTML và lưu vào database thông qua Mongoose.
- **Quản lý định tuyến và middleware** trong Express để tổ chức mã nguồn và xử lý các yêu cầu.
- **Xử lý lỗi và thông báo:** Cải thiện trải nghiệm người dùng bằng cách hiển thị thông báo thành công/thất bại sau các thao tác CRUD.

II. Các Chức Năng Chính

Ứng dụng sẽ có các chức năng quản lý bài viết blog cơ bản:

1. **Trang chủ:** Hiển thị danh sách tất cả các bài viết blog dưới dạng các "card" tóm tắt (tiêu đề, tác giả, ngày đăng, mô tả ngắn). Có thể có phân trang cơ bản (tùy chọn nâng cao).
2. **Xem chi tiết bài viết:** Nhấp vào một bài viết để xem toàn bộ nội dung, tác giả và ngày đăng.
3. **Thêm bài viết mới:** Form để người dùng nhập tiêu đề, tác giả, nội dung và mô tả ngắn cho bài viết.
4. **Chỉnh sửa bài viết:** Form để chỉnh sửa thông tin và nội dung của một bài viết hiện có.
5. **Xóa bài viết:** Nút để xóa một bài viết khỏi database.
6. **Thêm trường hợp đặc biệt (Tùy chọn nâng cao):**
 - Có thể thêm trường `isFeatured` (boolean) vào model bài viết.
 - Hiển thị các bài viết được đánh dấu `isFeatured: true` ở đầu trang hoặc trong một phần riêng.
 - Thêm nút/checkbox để chuyển đổi trạng thái `isFeatured` cho một bài viết.

III. Công Nghệ Sử Dụng

- **Backend:** Node.js, Express.js
- **Database:** MongoDB (yêu cầu cài đặt MongoDB Compass hoặc chạy MongoDB server cục bộ/trên đám mây)
- **ODM (Object Data Modeling):** Mongoose
- **Templating Engine:** Handlebars (với `express-handlebars`)
- **Styling (Tùy chọn):** CSS cơ bản để giao diện dễ nhìn.

IV. Cấu Trúc File Thư Mục

```
simple-blog-app/  
├── app.js                # File chính của ứng dụng Express  
├── package.json  
├── package-lock.json  
├── node_modules/  
├── config/  
│   └── db.js            # Cấu hình kết nối MongoDB  
└──
```


models/	# Chứa định nghĩa các Schema/Model của Mongoose
└─ Post.js	# Model cho bài viết blog
public/	# Chứa các file tĩnh (CSS, JS phía client)
└─ css/	
└─ style.css	# File CSS cơ bản
views/	# Thư mục chứa tất cả các file .handlebars
└─ layouts/	
└─ main.handlebars	# Layout chính của ứng dụng
└─ partials/	
└─ _header.handlebars	# Partial cho tiêu đề trang
└─ _footer.handlebars	# Partial cho chân trang
└─ _postCard.handlebars	# Partial cho một "card" bài viết trong danh sách
sách	
└─ index.handlebars	# Trang chủ hiển thị danh sách bài viết
└─ addPost.handlebars	# Trang form thêm bài viết mới
└─ editPost.handlebars	# Trang form chỉnh sửa bài viết
└─ postDetail.handlebars	# Trang hiển thị chi tiết bài viết
data/	# Thư mục chứa dữ liệu mẫu JSON
└─ posts.json	# Dữ liệu mẫu các bài viết để import vào MongoDB
README.md	

V. Tóm Tắt Danh Sách Các Bước Thực Hiện

1. **Thiết lập dự án & Cài đặt gói:** Khởi tạo Node.js, cài Express, Mongoose, Handlebars.
2. **Cấu hình kết nối MongoDB:** Tạo `config/db.js` để kết nối Mongoose với MongoDB.
3. **Tạo Mongoose Model:** Định nghĩa Schema và Model cho `Post` trong `models/Post.js`.
4. **Tạo dữ liệu mẫu JSON:** Tạo `data/posts.json` với một vài bài viết mẫu.
5. **Import dữ liệu mẫu vào MongoDB (thủ công hoặc script):** Sử dụng `mongoimport` hoặc một script Node.js.
6. **Cấu hình Express & Handlebars:** Trong `app.js`, thiết lập view engine, thư mục `views`, `layouts`, `partials` và `helpers`.
7. **Tạo cấu trúc thư mục View và file Layout chính (`main.handlebars`).**
8. **Tạo CSS cơ bản (`style.css`).**
9. **Xây dựng Trang Chủ (Read All Posts):**
 - Route `GET /`.
 - `index.handlebars` hiển thị danh sách bài viết dùng `{{#each}}`.
 - Partial `_postCard.handlebars` để hiển thị tóm tắt một bài viết.
10. **Xây dựng Chức năng Thêm Bài viết (Create Post):**
 - Route `GET /posts/add` (hiển thị form).
 - View `addPost.handlebars` (form thêm).
 - Route `POST /posts` (xử lý dữ liệu form, lưu vào DB).
11. **Xây dựng Chức năng Xem Chi tiết Bài viết (Read Single Post):**
 - Route `GET /posts/:id`.
 - View `postDetail.handlebars`.
12. **Xây dựng Chức năng Chỉnh sửa Bài viết (Update Post):**
 - Route `GET /posts/edit/:id` (hiển thị form với dữ liệu có sẵn).
 - View `editPost.handlebars` (form chỉnh sửa).
 - Route `POST /posts/edit/:id` (xử lý cập nhật vào DB).
13. **Xây dựng Chức năng Xóa Bài viết (Delete Post):**
 - Route `POST /posts/delete/:id`.
14. **Thêm Custom Helpers (ví dụ: định dạng ngày tháng).**
15. **Thêm thông báo (Flash Messages) với `connect-flash` và `express-session` (Tùy chọn nâng cao).**
16. **Kiểm tra và Debug.**

VI. Chi Tiết Các Bước Thực Hiện

Bước 1: Thiết lập dự án & Cài đặt gói

1. Mở terminal:

Bash

```
mkdir simple-blog-app
cd simple-blog-app
npm init -y
npm install express express-handlebars mongoose connect-flash express-session # Thêm connect-flash và express-session cho thông báo
```

Bước 2: Cấu hình kết nối MongoDB (config/db.js)

Tạo thư mục config và file config/db.js:

JavaScript

```
// config/db.js
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI ||
    'mongodb://localhost:27017/simpleblog', {
      // Các tùy chọn Mongoose được khuyến nghị (tùy thuộc phiên bản
      // Mongoose của bạn, có thể không cần nữa)
      // useNewUrlParser: true,
      // useUnifiedTopology: true,
      // useCreateIndex: true, // Nếu bạn dùng index
      // useFindAndModify: false // Nếu bạn dùng findByIdAndUpdate
    });
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (err) {
    console.error(`Error: ${err.message}`);
    process.exit(1); // Thoát ứng dụng nếu kết nối lỗi
  }
};

module.exports = connectDB;
```

Lưu ý: mongodb://localhost:27017/simpleblog là URI mặc định. Đảm bảo MongoDB server của bạn đang chạy.

Bước 3: Tạo Mongoose Model (models/Post.js)

Tạo thư mục models và file models/Post.js:

JavaScript

```
// models/Post.js
const mongoose = require('mongoose');

const PostSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
    trim: true // Loại bỏ khoảng trắng ở đầu và cuối
  },
  // ... các trường khác
});
```

```

    author: {
      type: String,
      required: true,
      trim: true
    },
    content: {
      type: String,
      required: true
    },
    description: { // Mô tả ngắn cho card bài viết
      type: String,
      required: true,
      maxlength: 200 // Giới hạn độ dài mô tả
    },
    isFeatured: { // Trường tùy chọn nâng cao
      type: Boolean,
      default: false
    },
    createdAt: {
      type: Date,
      default: Date.now // Tự động thêm ngày tạo
    }
  }
});

module.exports = mongoose.model('Post', PostSchema);

```

Bước 4: Tạo dữ liệu mẫu JSON (data/posts.json)

Tạo thư mục data và file data/posts.json:

JSON

```

[
  {
    "title": "Giới thiệu về Node.js và Express",
    "author": "Nguyễn Văn A",
    "content": "Node.js là một môi trường chạy JavaScript mã nguồn mở, đa nền tảng, được xây dựng trên công cụ JavaScript V8 của Google Chrome. Express.js là một framework ứng dụng web tối giản và linh hoạt cho Node.js, cung cấp một bộ tính năng mạnh mẽ để phát triển các ứng dụng web và API.",
    "description": "Tìm hiểu những điều cơ bản về Node.js và Express.js, hai công nghệ quan trọng để xây dựng ứng dụng web phía máy chủ.",
    "isFeatured": true,
    "createdAt": "2024-01-15T10:00:00.000Z"
  },
  {
    "title": "MongoDB và Mongoose: Cẩm nang toàn diện",
    "author": "Trần Thị B",
    "content": "MongoDB là một cơ sở dữ liệu NoSQL dựa trên tài liệu, cung cấp khả năng mở rộng cao và hiệu suất linh hoạt. Mongoose là một thư viện ODM (Object Data Modeling) cho MongoDB và Node.js, giúp việc thao tác với dữ liệu trở nên dễ dàng và có cấu trúc hơn.",
    "description": "Đi sâu vào cách MongoDB hoạt động và cách Mongoose giúp bạn tương tác với cơ sở dữ liệu từ ứng dụng Node.js.",
    "isFeatured": false,
    "createdAt": "2024-02-20T11:30:00.000Z"
  },
  {
    "title": "Làm chủ Templating với Handlebars.js",
    "author": "Lê Văn C",
    "content": "Handlebars.js là một công cụ tạo khuôn mẫu JavaScript mạnh mẽ, giúp bạn tạo ra các trang web động một cách hiệu quả. Nó tách biệt logic

```

```

    ứng dụng khỏi phần trình bày (HTML), giúp mã dễ đọc, dễ bảo trì và mở rộng
    hơn.",
    "description": "Khám phá cách sử dụng Handlebars để tạo các giao diện
    động, sử dụng layouts, partials và custom helpers.",
    "isFeatured": true,
    "createdAt": "2024-03-10T09:15:00.000Z"
  },
  {
    "title": "Xây dựng RESTful API với Express",
    "author": "Phạm Thị D",
    "content": "RESTful API là một kiến trúc tiêu chuẩn để xây dựng các dịch
    vụ web. Bài viết này sẽ hướng dẫn bạn từng bước cách tạo một API RESTful mạnh
    mẽ bằng Express.js, xử lý các phương thức HTTP khác nhau và quản lý tài
    nguyên.",
    "description": "Hướng dẫn chi tiết về cách thiết kế và triển khai API
    RESTful sử dụng framework Express của Node.js.",
    "isFeatured": false,
    "createdAt": "2024-04-01T14:00:00.000Z"
  }
]

```

Bước 5: Import dữ liệu mẫu vào MongoDB

Bạn có hai cách:

- **Cách 1 (Thủ công với mongoimport - khuyến nghị cho bài tập này):**

1. Mở terminal.
2. Đảm bảo MongoDB server đang chạy.
3. Di chuyển đến thư mục gốc của dự án (simple-blog-app).
4. Chạy lệnh import:

Bash

```
mongoimport --db simpleblog --collection posts --file
data/posts.json --jsonArray
```

- --db simpleblog: Tên database của bạn.
- --collection posts: Tên collection mà dữ liệu sẽ được import vào (phải trùng với tên Model của Mongoose).
- --file data/posts.json: Đường dẫn đến file JSON của bạn.
- --jsonArray: Chỉ định rằng file JSON là một mảng các đối tượng.

5. Bạn có thể kiểm tra trong MongoDB Compass để xem dữ liệu đã được import thành công chưa.

- **Cách 2 (Sử dụng script Node.js - nâng cao hơn):** Bạn có thể tạo một script riêng để import dữ liệu chương trình. Đây là một file importData.js bạn có thể thêm vào thư mục gốc:

JavaScript

```
// importData.js
const mongoose = require('mongoose');
const dotenv = require('dotenv'); // Nếu bạn muốn dùng .env cho MONGO_URI
const Post = require('./models/Post');
const posts = require('./data/posts.json'); // Import dữ liệu JSON
```

```

dotenv.config({ path: './config/config.env' }); // Hoặc cấu hình khác
nếu cần

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI ||
'mongodb://localhost:27017/simpleblog');
    console.log('MongoDB Connected for import!');
  } catch (err) {
    console.error(`MongoDB connection error: ${err.message}`);
    process.exit(1);
  }
};

const importData = async () => {
  try {
    await Post.deleteMany(); // Xóa tất cả dữ liệu cũ trước khi
import
    await Post.insertMany(posts); // Thêm dữ liệu mới
    console.log('Data Imported!');
    process.exit();
  } catch (err) {
    console.error(`Error importing data: ${err.message}`);
    process.exit(1);
  }
};

connectDB();
importData();

```

Để chạy script này, bạn sẽ dùng lệnh: `node importData.js` (sau khi cài dotenv nếu cần).

Bước 6: Cấu hình Express & Handlebars trong app.js

Cập nhật app.js (tiếp tục từ mã ở trên):

JavaScript

```

// app.js
const express = require('express');
const { engine } = require('express-handlebars');
const path = require('path');
const connectDB = require('./config/db'); // Import hàm kết nối DB
const Post = require('./models/Post'); // Import Post Model
const session = require('express-session'); // Cho flash messages
const flash = require('connect-flash'); // Cho flash messages

const app = express();
const PORT = process.env.PORT || 3000;

// Connect to Database
connectDB();

// --- Cấu hình Handlebars ---
app.engine('handlebars', engine({
  defaultLayout: 'main',
  layoutsDir: path.join(__dirname, 'views/layouts'),
  partialsDir: path.join(__dirname, 'views/partials'),
  extname: '.handlebars',
  helpers: {

```

```

// Custom Helpers
formatDate: (date) => { // Định dạng ngày tháng
    return new Date(date).toLocaleDateString('vi-VN', {
        year: 'numeric',
        month: 'long',
        day: 'numeric'
    });
},
truncate: (str, len) => { // Cắt ngắn chuỗi
    if (str.length > len) {
        return str.substring(0, len) + '...';
    }
    return str;
},
isFeatured: (status) => status ? 'Có' : 'Không', // Helper cho trạng
thái nổi bật
// Helper so sánh cho checkbox (sẽ dùng trong form edit)
isChecked: (value) => value ? 'checked' : ''
}
});
app.set('view engine', 'handlebars');
app.set('views', path.join(__dirname, 'views'));

// --- Middleware ---
app.use(express.static(path.join(__dirname, 'public'))); // Phục vụ file tĩnh
app.use(express.urlencoded({ extended: true })); // Xử lý dữ liệu form (x-
www-form-urlencoded)
app.use(express.json()); // Xử lý dữ liệu JSON (nếu có)

// Express Session Middleware (cho connect-flash)
app.use(session({
    secret: 'supersecretkey', // Chuỗi bí mật để mã hóa session
    resave: false,
    saveUninitialized: false
}));

// Connect Flash Middleware
app.use(flash());

// Middleware để đưa flash messages vào res.locals (có thể truy cập trong
Handlebars)
app.use((req, res, next) => {
    res.locals.success_msg = req.flash('success_msg');
    res.locals.error_msg = req.flash('error_msg');
    res.locals.error = req.flash('error'); // Dùng cho passport nếu có
    next();
});

// --- Routes (sẽ thêm bên dưới) ---

// --- Khởi động Server ---
app.listen(PORT, () => {
    console.log(`Server đang chạy tại http://localhost:${PORT}`);
});

```

Bước 7: Tạo cấu trúc thư mục View và file Layout chính (main.handlebars)

Tạo các thư mục và file theo cấu trúc đã định nghĩa ở mục IV.

Tạo views/layouts/main.handlebars:

HTML

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{{title}} - Blog Của Tôi</title>
  <link rel="stylesheet" href="/css/style.css">
</head>
<body>
  <header class="app-header">
    <h1><a href="/">Blog Của Tôi</a></h1>
    <nav>
      <a href="/">Trang chủ</a> |
      <a href="/posts/add">Thêm bài viết</a>
    </nav>
  </header>

  <main class="container">
    {{#if success_msg}}
      <div class="alert alert-success">{{success_msg}}</div>
    {{/if}}
    {{#if error_msg}}
      <div class="alert alert-danger">{{error_msg}}</div>
    {{/if}}
    {{#if error}} <div class="alert alert-danger">{{error}}</div>
    {{/if}}

    {{{body}}} </main>

  <footer class="app-footer">
    <p>&copy; {{getCurrentYear}} Blog Của Tôi. All rights reserved.</p>
  </footer>
</body>
</html>
```

Lưu ý: Thêm {{getCurrentYear}} vào helpers trong app.js nếu chưa có.

Bước 8: Tạo CSS cơ bản (public/css/style.css)

Bạn có thể tự tạo hoặc sử dụng CSS đã cung cấp trong ví dụ trước (cho Task Manager) và điều chỉnh lại cho phù hợp với Blog Post (ví dụ: post-card, post-detail-card).

Bước 9: Xây dựng Trang Chủ (Read All Posts)

Thêm route vào app.js:

JavaScript

```
// Route Trang Chủ: Hiển thị tất cả bài viết
app.get('/', async (req, res) => {
  try {
    const posts = await Post.find().sort({ createdAt: 'desc' }).lean();
    // .lean() để chuyển Mongoose document thành plain JS object
    res.render('index', { title: 'Tất cả bài viết', posts: posts });
  } catch (err) {
    console.error(err);
    res.render('index', { title: 'Lỗi', error_msg: 'Không thể tải bài viết.' });
  }
});
```

```

    }
  });

```

Tạo views/index.handlebars:

HTML

```

<div class="post-grid">
  {{#each posts}}
    {{> _postCard this}} {{else}}
      <p>Chưa có bài viết nào. Hãy thêm một bài viết mới!</p>
    {{/each}}
</div>

```

Tạo views/partials/_postCard.handlebars:

HTML

```

<div class="post-card {{#if isFeatured}}featured-post{{/if}}">
  <h3><a href="/posts/{{_id}}">{{title}}</a></h3>
  <p class="author">Tác giả: {{author}}</p>
  <p class="date">Ngày đăng: {{formatDate createdAt}}</p>
  <p class="description">{{truncate description 150}}</p>
  <div class="post-actions">
    <a href="/posts/edit/{{_id}}" class="edit-btn">Sửa</a>
    <form action="/posts/delete/{{_id}}" method="POST"
      style="display:inline;">
      <button type="submit" class="delete-btn">Xóa</button>
    </form>
  </div>
</div>

```

Bước 10: Xây dựng Chức năng Thêm Bài viết (Create Post)

Thêm route vào app.js:

JavaScript

```

// Route hiển thị form thêm bài viết
app.get('/posts/add', (req, res) => {
  res.render('addPost', { title: 'Thêm Bài viết Mới' });
});

// Route xử lý thêm bài viết mới
app.post('/posts', async (req, res) => {
  const { title, author, content, description } = req.body;
  let errors = [];

  if (!title || !author || !content || !description) {
    errors.push({ msg: 'Vui lòng điền đầy đủ tất cả các trường.' });
  }

  if (errors.length > 0) {
    res.render('addPost', {
      title: 'Thêm Bài viết Mới',
      errors: errors,
      title: title, // Giữ lại dữ liệu đã nhập
      author: author,
      content: content,
      description: description
    });
  }
});

```



```

    } else {
      try {
        const newPost = new Post({
          title,
          author,
          content,
          description,
          isFeatured: req.body.isFeatured === 'on' ? true : false // Xử
lý checkbox
        });
        await newPost.save();
        req.flash('success_msg', 'Bài viết đã được thêm thành công!');
        res.redirect('/');
      } catch (err) {
        console.error(err);
        req.flash('error_msg', 'Có lỗi xảy ra khi thêm bài viết. ');
        res.redirect('/posts/add'); // Quay lại form nếu có lỗi
      }
    }
  });
});

```

Tạo views/addPost.handlebars:

HTML

```

<h2>Thêm bài viết mới</h2>
{{#if errors}}
  <div class="alert alert-danger">
    {{#each errors}}
      <p>{{this.msg}}</p>
    {{/each}}
  </div>
{{/if}}
<form action="/posts" method="POST">
  <div>
    <label for="title">Tiêu đề:</label>
    <input type="text" id="title" name="title" value="{{title}}"
required>
  </div>
  <div>
    <label for="author">Tác giả:</label>
    <input type="text" id="author" name="author" value="{{author}}"
required>
  </div>
  <div>
    <label for="description">Mô tả ngắn (tối đa 200 ký tự):</label>
    <textarea id="description" name="description" rows="3"
maxlength="200" required>{{description}}</textarea>
  </div>
  <div>
    <label for="content">Nội dung bài viết:</label>
    <textarea id="content" name="content" rows="10"
required>{{content}}</textarea>
  </div>
  <div>
    <label for="isFeatured">
      <input type="checkbox" id="isFeatured" name="isFeatured" {{#if
isFeatured}}checked{{/if}}>
      Bài viết nổi bật
    </label>
  </div>
  <div>
    <button type="submit">Thêm bài viết</button>
  </div>
</form>

```

```
</div>
</form>
```

Bước 11: Xây dựng Chức năng Xem Chi tiết Bài viết (Read Single Post)

Thêm route vào app.js:

JavaScript

```
// Route xem chi tiết bài viết
app.get('/posts/:id', async (req, res) => {
  try {
    const post = await Post.findById(req.params.id).lean();
    if (post) {
      res.render('postDetail', { title: post.title, post: post });
    } else {
      req.flash('error_msg', 'Không tìm thấy bài viết.');
```

Tạo views/postDetail.handlebars:

HTML

```
<div class="post-detail-card">
  <h2>{{post.title}}</h2>
  <p class="author">Tác giả: {{post.author}}</p>
  <p class="date">Ngày đăng: {{formatDate post.createdAt}}</p>
  <hr>
  <div class="post-content">
    <p>{{post.content}}</p>
  </div>
  <div class="post-meta">
    <p>Bài viết nổi bật: <strong>{{isFeatured
post.isFeatured}}</strong></p>
  </div>
  <div class="post-actions">
    <a href="/posts/edit/{{post._id}}" class="edit-btn">Sửa bài viết</a>
    <form action="/posts/delete/{{post._id}}" method="POST"
style="display:inline; margin-left:10px;">
      <button type="submit" class="delete-btn">Xóa bài viết</button>
    </form>
    <a href="/" style="margin-left:10px;">Quay lại danh sách</a>
  </div>
</div>
```

Bước 12: Xây dựng Chức năng Chỉnh sửa Bài viết (Update Post)

Thêm route vào app.js:

JavaScript

```
// Route hiển thị form chỉnh sửa bài viết
app.get('/posts/edit/:id', async (req, res) => {
```

```

    try {
      const post = await Post.findById(req.params.id).lean();
      if (post) {
        res.render('editPost', { title: `Sửa: ${post.title}`, post: post
});
      } else {
        req.flash('error_msg', 'Không tìm thấy bài viết để sửa.');
```

```

        res.redirect('/');
      }
    } catch (err) {
      console.error(err);
      req.flash('error_msg', 'Có lỗi xảy ra khi tải bài viết để sửa.');
```

```

      res.redirect('/');
    }
  });

// Route xử lý cập nhật bài viết
app.post('/posts/edit/:id', async (req, res) => {
  const postId = req.params.id;
  const { title, author, content, description } = req.body;
  const isFeatured = req.body.isFeatured === 'on' ? true : false;
  let errors = [];

  if (!title || !author || !content || !description) {
    errors.push({ msg: 'Vui lòng điền đầy đủ tất cả các trường.' });
  }

  if (errors.length > 0) {
    // Giữ lại dữ liệu cũ nếu có lỗi
    const oldPost = await Post.findById(postId).lean();
    res.render('editPost', {
      title: `Sửa: ${oldPost ? oldPost.title : 'Bài viết'}`,
      post: { ...oldPost, title, author, content, description,
isFeatured }, // Kết hợp dữ liệu cũ và mới nhập
      errors: errors
    });
  } else {
    try {
      await Post.findByIdAndUpdate(postId, {
        title,
        author,
        content,
        description,
        isFeatured
      }, { new: true, runValidators: true }); // new: true trả về doc
mới, runValidators: true chạy lại validation
      req.flash('success_msg', 'Bài viết đã được cập nhật thành
công!');
      res.redirect(`/posts/${postId}`);
    } catch (err) {
      console.error(err);
      req.flash('error_msg', 'Có lỗi xảy ra khi cập nhật bài viết.');
```

```

      res.redirect(`/posts/edit/${postId}`);
    }
  }
});

```

Tạo views/editPost.handlebars:

HTML

```

<h2>Sửa bài viết: {{post.title}}</h2>
{{#if errors}}

```

```

    <div class="alert alert-danger">
      {{#each errors}}
        <p>{{this.msg}}</p>
      {{/each}}
    </div>
  {{/if}}
<form action="/posts/edit/{{post._id}}" method="POST">
  <div>
    <label for="title">Tiêu đề:</label>
    <input type="text" id="title" name="title" value="{{post.title}}"
required>
  </div>
  <div>
    <label for="author">Tác giả:</label>
    <input type="text" id="author" name="author" value="{{post.author}}"
required>
  </div>
  <div>
    <label for="description">Mô tả ngắn (tối đa 200 ký tự):</label>
    <textarea id="description" name="description" rows="3"
maxlength="200" required>{{post.description}}</textarea>
  </div>
  <div>
    <label for="content">Nội dung bài viết:</label>
    <textarea id="content" name="content" rows="10"
required>{{post.content}}</textarea>
  </div>
  <div>
    <label for="isFeatured">
      <input type="checkbox" id="isFeatured" name="isFeatured"
{{ifChecked post.isFeatured}}>
      Bài viết nổi bật
    </label>
  </div>
  <div>
    <button type="submit">Cập nhật bài viết</button>
  </div>
</form>

```

Bước 13: Xây dựng Chức năng Xóa Bài viết (Delete Post)

Thêm route vào app.js:

JavaScript

```

// Route xử lý xóa bài viết
app.post('/posts/delete/:id', async (req, res) => {
  try {
    const deletedPost = await Post.findByIdAndDelete(req.params.id);
    if (deletedPost) {
      req.flash('success_msg', 'Bài viết đã được xóa thành công!');
      res.redirect('/');
    } else {
      req.flash('error_msg', 'Không tìm thấy bài viết để xóa.');
```

Bước 14: Thêm Custom Helpers (Đã thêm trong Bước 6)

Kiểm tra lại `app.js` để đảm bảo các helper `formatDate`, `truncate`, `isFeatured`, `ifChecked` đã được định nghĩa.

Bước 15: Thêm thông báo (Flash Messages) (Đã thêm trong Bước 6)

Kiểm tra lại `app.js` để đảm bảo middleware `express-session` và `connect-flash` đã được thiết lập, và `res.locals` được gán các flash messages. Các `div alert` đã được thêm vào `main.handlebars`.

Bước 16: Chạy ứng dụng và kiểm tra

1. Đảm bảo MongoDB server đang chạy.
2. Chạy `node app.js` trong terminal từ thư mục gốc của dự án.
3. Truy cập `http://localhost:3000` trong trình duyệt.

VII. 03 Test Case

Để kiểm tra các chức năng chính của ứng dụng:

1. Test Case 1: Thêm bài viết mới và kiểm tra hiển thị.

○ Các bước:

1. Truy cập `/posts/add`.
2. Nhập đầy đủ thông tin cho một bài viết mới (ví dụ: Tiêu đề: "Bài viết mới", Tác giả: "Tôi", Mô tả: "Đây là bài viết đầu tiên.", Nội dung: "Nội dung chi tiết của bài viết đầu tiên."). Có thể chọn hoặc bỏ chọn "Bài viết nổi bật".
3. Nhấn nút "Thêm bài viết".

○ Kết quả mong đợi:

1. Bạn được chuyển hướng về trang chủ (/).
2. Một thông báo "Bài viết đã được thêm thành công!" xuất hiện.
3. Bài viết mới của bạn xuất hiện trong danh sách trên trang chủ.

2. Test Case 2: Chỉnh sửa bài viết hiện có.

○ Các bước:

1. Trên trang chủ, tìm một bài viết đã có (ví dụ: "Giới thiệu về Node.js và Express").
2. Nhấp vào nút "Sửa" bên cạnh bài viết đó (hoặc vào chi tiết bài viết rồi nhấn nút sửa).
3. Thay đổi Tiêu đề thành "Giới thiệu Node.js và Express Nâng Cao", Mô tả thành "Khám phá sâu hơn về Node.js và Express." và bỏ chọn "Bài viết nổi bật" (nếu nó đang nổi bật).
4. Nhấn nút "Cập nhật bài viết".

○ Kết quả mong đợi:

1. Bạn được chuyển hướng về trang chi tiết của bài viết đó (`/posts/:id`).
2. Thông báo "Bài viết đã được cập nhật thành công!" xuất hiện.
3. Thông tin của bài viết (tiêu đề, mô tả, trạng thái nổi bật) đã được cập nhật chính xác.

3. Test Case 3: Xóa một bài viết.

○ Các bước:

1. Trên trang chủ, tìm một bài viết bất kỳ (ví dụ: "MongoDB và Mongoose: Cẩm nang toàn diện").
2. Nhấn nút "Xóa" bên cạnh bài viết đó.

○ Kết quả mong đợi:

1. Bạn được chuyển hướng về trang chủ (/).
2. Thông báo "Bài viết đã được xóa thành công!" xuất hiện.
3. Bài viết đó biến mất khỏi danh sách trên trang chủ.

PHÂN TÍCH QUY LUẬT THỰC THI

PROMT: Tôi là người mới học, tôi cần phát hiện quy luật thực thi của chương trình. Hãy Mô tả quá trình thực thi của code của bài tập

BÀI TẬP : Quản lý Danh sách Khóa học"

Mục tiêu bài tập

- Hiểu cách tổ chức ứng dụng Node.js với Express.
- Sử dụng **MongoDB + Mongoose** để thao tác dữ liệu.
- Áp dụng **Handlebars** để tạo giao diện động.
- Tích hợp và phân tách rõ ràng giữa **logic (controller)** và **giao diện (views)**.

Chức năng yêu cầu

1. ☒ Hiển thị danh sách các khóa học từ MongoDB.
2. ☒ Thêm khóa học mới qua form (sử dụng POST).
3. ☒ Xóa khóa học theo ID.
4. ☒ (Bonus) Hiển thị chi tiết khóa học khi click.

Công nghệ sử dụng

- **Node.js**
- **Express**
- **MongoDB**
- **Mongoose**
- **Express-Handlebars (hbs)**

Cấu trúc thư mục đề xuất

pgsql

```
course-manager/
├── package.json
├── app.js
├── config/
│   └── db.js
├── models/
│   └── Course.js
├── routes/
│   └── courseRoutes.js
├── controllers/
│   └── courseController.js
├── views/
│   ├── layouts/
│   │   └── main.handlebars
│   ├── courses/
│   │   ├── index.handlebars
│   │   ├── detail.handlebars
│   │   └── add.handlebars
│   └── partials/
│       └── header.handlebars
└── public/
    └── css/
        └── style.css
```

```
└─ seed/
   └─ courses.json
```

Tóm tắt các bước thực hiện

Bước 1: Khởi tạo project và cài gói cần thiết

```
bash

npm init -y
npm install express mongoose express-handlebars body-parser
```

Bước 2: Cấu hình kết nối MongoDB (file `config/db.js`)

```
js

const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect('mongodb://localhost:27017/course-manager', {
      useNewUrlParser: true,
      useUnifiedTopology: true
    });
    console.log('MongoDB connected!');
  } catch (err) {
    console.error(err);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Bước 3: Tạo model Course (file `models/Course.js`)

```
js

const mongoose = require('mongoose');

const CourseSchema = new mongoose.Schema({
  title: String,
  description: String,
  instructor: String,
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Course', CourseSchema);
```

Bước 4: Tạo controller logic (file `controllers/courseController.js`)

```
js

const Course = require('../models/Course');

exports.getAllCourses = async (req, res) => {
  const courses = await Course.find();
  res.render('courses/index', { courses });
};
```



```

exports.getAddForm = (req, res) => {
  res.render('courses/add');
};

exports.createCourse = async (req, res) => {
  const { title, description, instructor } = req.body;
  await Course.create({ title, description, instructor });
  res.redirect('/');
};

exports.deleteCourse = async (req, res) => {
  await Course.findByIdAndDelete(req.params.id);
  res.redirect('/');
};

```

Bước 5: Tạo router (file routes/courseRoutes.js)

```

js

const express = require('express');
const router = express.Router();
const courseController = require('../controllers/courseController');

router.get('/', courseController.getAllCourses);
router.get('/add', courseController.getAddForm);
router.post('/add', courseController.createCourse);
router.get('/delete/:id', courseController.deleteCourse);

module.exports = router;

```

Bước 6: Cấu hình app chính (app.js)

```

js

const express = require('express');
const exphbs = require('express-handlebars');
const bodyParser = require('body-parser');
const connectDB = require('./config/db');
const courseRoutes = require('./routes/courseRoutes');

const app = express();

// Connect to DB
connectDB();

// Middleware
app.use(bodyParser.urlencoded({ extended: false }));
app.use(express.static('public'));

// View Engine
app.engine('handlebars', exphbs());
app.set('view engine', 'handlebars');

// Routes
app.use('/', courseRoutes);

const PORT = 3000;
app.listen(PORT, () => console.log(`Server running on http://localhost:${PORT}`));

```

Test Case

#	Mô tả kiểm thử	Input	Mong đợi
1	Truy cập /	-	Hiển thị danh sách khóa học từ MongoDB
2	Thêm khóa học mới	Form: title + description + instructor	Sau khi submit, khóa học mới xuất hiện trong danh sách
3	Xóa khóa học	Nhấn nút "Xóa" tại 1 khóa học	Khóa học đó bị xóa và không còn hiển thị

Dữ liệu mẫu (seed/courses.json)

json

```
[
  {
    "title": "Lập trình Node.js cơ bản",
    "description": "Học cách tạo ứng dụng với Node và Express.",
    "instructor": "Nguyễn Văn A"
  },
  {
    "title": "MongoDB cho người mới bắt đầu",
    "description": "Lưu trữ và truy vấn dữ liệu với MongoDB.",
    "instructor": "Trần Thị B"
  }
]
```

Bạn có thể import dữ liệu này bằng MongoDB Compass hoặc dùng lệnh:

bash

```
mongoimport --db course-manager --collection courses --file seed/courses.json --jsonArray
```